

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației***

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NANESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NANESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și management-ul contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm/VS Code și biblioteci specifice precum LangChain, LangGraph sau GoogleADK, concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NANESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NANESCU

.....

Cuprins

LISTA FIGURILOR	2
LISTA TABELELOR	4
LISTA DE ACRONIME	5
INTRODUCERE	6
CONTEXT SI MOTIVATIE	6
SCOPUL SI OBIECTIVELE LUCRARI	7
1. LUCRARI ANTERIOARE	8
1.1. Drumul catre modelele mari de limbaj (Large Language Model (LLM))	8
1.2. Maturizarea modelelor mari de limbaj si cercetari actuale	13
1.3. Solutii agentice pentru industria dezvoltarii software	19
1.4. Necesitati pentru dezvoltarea unei solutii agentice in domeniul dezvoltarii software	27
1.5. Puncte slabe ale arhitecturilor agentice	27
1.6. Alegerea solutiei de implementare pe baza lucrarilor studiate si a standardelor din industrie	28
2. PLASAREA IN INDUSTRIE	32
2.1. Open source software	32
2.2. Flux agentice ce ruleaza local	32
2.3. Observability	33
2.4. Securitate si confidentialitate	33
2.5. Cost	33
2.6. Aplicatii rivale	34
2.7. Mediul de dezvoltare	35
2.7.1. Cerinte hardware	35
2.7.2. Cerinte software	36
2.8. Structura	36
2.9. Configurare	37
2.10. Interfata cu utilizatorul si instructiuni de utilizare	37
2.11. Management-ul contextului si al memoriei	41
2.11.1. Memoria	41
2.11.2. Contextul	41
2.12. Subagenti, Unelte si capabilitati	42
2.12.1. Tools	42
2.12.2. Skills	42
2.12.3. Subagents	42
2.13. Extensibilitate	42
2.13.1. Model Context Protocol (MCP)	42
2.13.2. Subagenti si arhitectura agnostica de furnizori	42

2.14. Securitate	42
2.14.1. Sandboxing	42
2.14.2. Guardrails, Out of Scope (OOS) and dangerous intent	42
2.14.3. Tool execution order and logic	42
2.15. Observabilitate	42
2.15.1. Langfuse	42
2.15.2. Panou web	42
3. REZULTATE EXPERIMENTALE	45
3.1. Experimente, evals si benchmarking	45
3.2. Contextul in mediul agentic	45
3.3. Modele vs context si unelte	45
4. CONCLUZII	46
5. IMBUNATATIRI VIITOARE	47
5.1. Automatizari, integrari cu alte programe	47
BIBLIOGRAFIE	48

LISTA FIGURILOR

1.1. Modelul unui perceptron	8
1.2. Convolutia cu un nucleu a unei imagini	10
1.3. Recurrent Neural Network (RNN) si Long Short Term Memory (LSTM) comparate	11
1.4. Arhitectura transformer	12
1.5. Evolutia recenta a modelelor mari de limbaj [1]	13
1.6. Evolutia modelelor GPT [1]	13
1.7. Performanta tipurilor de arhitecturi de tipul "Longformer" [2]. In timp ce meca- nismul tipic de atentie utilizeaza memorie exponential, arhitectura Longformer foloseste memorie liniar odata cu cresterea contextului	14
1.8. Diferite configuratii de atentie folosite in arhitectura Longformer in comparatie cu mecanismul obisnuit de atentie [2]	14
1.9. Principiul de baza pentru Rotary Position Embedding [3]	15
1.10. Fine-tuning-ul se realizeaza doar pe A si B [4]	15
1.11. Rezultatele procesului de Simple Self Distillation [5]	16
1.12. Imbunatatiri in sarcini de programare ale modelelor dupa ce au fost supuse pro- cesului de Simple Self Distillation[5]	17
1.13. Comparatia procedeele de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [6]	17
1.14. Ilustrarea procedeeului de Reinforcement Learning (RL) [7]	18
1.15. Exemple de Chain-of-Thought (CoT) [8]	19
1.16. Diferenta intre procedee de gandire [9]	19
1.17. Paradigma ReAct [10]	20
1.18. Imbunatatirile pe care le aduce ReAct [10]	20
1.19. Ideea din spatele ReWOO [11]	21
1.20. Comparatia de performante intre paradigme pentru imbunatatirea gandirii din punctul de vedere al acuratetii vs. numarul de tokeni consumati [11]	21
1.21. Imbunatatirea adusa de ReWOO este delegarea mai multor libertati catre worker ce poate apela mai multe unelte pana sa produca un raspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [11]	22
1.22. Modele tind sa aiba o acuratete foarte slaba contra intrebarilor ce vizeaza infor- matii care se afla la mijlocul ferestrei curente de context [12]	23
1.23. Vizualizarea problemei si al rezultatului dorit la intrebare. De multe ori, modelele intampina dificultati in accesarea acestor informatii ascunse in mijlocul ferestrei de context [12]	23
1.24. In majoritatea modelelor se observa o scadere a performantelor dupa 32K tokeni pe sarcini de gandire complexa[13]	24
1.25. Clasamentul modelelor in SWE-Bench Pro [14] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza	26
1.26. Clasamentul modelelor in SWE-Bench Pro [14] la momentul actual in functie de cost	27
1.27. Reprezentarea ferestrei de context intr-o conversatie uzuala (sursa)	28
2.1. Reprezentarea ferestrei de context intr-o conversatie in Claude Code	33

2.2.	Reprezentarea utilizarii in Claude (varianta web) (sursa)	33
2.3.	Structura directoarelor vitale ale proiectului	36
2.4.	Prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual	37
2.5.	Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual	37
2.6.	A doua versiune a interfetei, realizata fara biblioteca Textual	38
2.7.	Continuarea conversatiei cu a doua versiune a interfetei, realizata fara biblioteca Textual	38
2.8.	Impartirea ferestrei de context (sursa)	41

LISTA TABELELOR

1.1.	Comparatia catorva din modelele existente si a capabilitatile acestora in comparatie cu pretul de utilizare	25
1.2.	Comparație framework-uri pentru workflow-uri agentice	29
1.3.	Comparație biblioteci pentru interfața cu utilizatorul	30
2.2.	Top 6 asistenti de programare open source	34
2.4.	Comenzi disponibile in interfata	40
2.1.	Produse AI ale principalilor furnizori de modele de limbaj	43
2.3.	Hardware recomandat pentru modele open-source	44

LISTA DE ACRONIME

API Application Programming Interface	MHA Multi-Head Attention
BTT Backpropagation Through Time	NLP Natural Language Processing
CD Continuous Deployment	OOS Out of Scope
CI Continuous Integration	OSS Open Source Software
CNN Convolutional Neural Network	RAG Retrieval Augmented Generation
CoT Chain-of-Thought	RCA Root Cause Analysis
E2E End to End	RL Reinforcement Learning
GPU Graphics Processing Unit	RNN Recurrent Neural Network
IA Inteligență Artificială	SDD Spec-Driven Development
KV Key-Value	TDD Test-Driven Development
LLM Large Language Model	ToT Tree-of-Thought
LSTM Long Short Term Memory	TUI Terminal User Interface
MCP Model Context Protocol	

INTRODUCERE

CONTEXT SI MOTIVATIE

Motivatia acestei lucrari vine din experienta personala in industria dezvoltarii software. Atributiile unui inginer software sunt numeroase, diverse si adesea repetitive. Aceasta congestie a timpului ingreuneaza procesul de dezvoltare. A scrie cod a devenit doar o mica parte din sarcinile unui inginer software, pierzand teren in fata design-urilor, documentatiei, review-urilor, a testelor unitare de integrare si End to End (E2E), instrumentarii si monitorizarii, procesului de Continuous Integration (CI)/Continuous Deployment (CD), infrastructurii, iar in cazul problemelor a debugging-ului si a analizei cauzelor principale (Root Cause Analysis (RCA)).

Un inginer software care lucreaza intr-o companie ce deservește milioane de clienti din intreaga lume trebuie sa fie competitiv, iar aceasta competitivitate se rezuma la a fi la curent cu cele mai noi practici din domeniu, cunoasterea notiunilor de programare, securitate cibernetica, retelistica si infrastructura, dar sa si retina simultan informatii vitale despre arhitectura si sistemele interne ale companiei, totul pe langa instruirea colegilor noi sau sedinte.

Timpul unui inginer este astfel impartit de foarte multe procese dinamice care trebuiesc adresate de un angajat capabil sa sustina acest ritm alert.

Avand in vedere cele de mai sus, cat si evolutia recenta a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) pentru intelegerea si generarea de text, era natural ca directia in care dezvoltarea software urma sa se schimbe catre un proces in care multe din sarcinile pe care un inginer le-ar fi avut sunt acum delegate sistemelor automate. Astfel, atentia inginerului se muta de la a scrie cod, la supravegherea instrumentelor de programare bazate pe inteligenta artificiala si asumarea deciziilor arhitecturale. Restul grijilor cotidiene pot fi abordate de aceste instrumente, eliberand programul operatorului.

Trecerea de la o experienta pur conversationala cu aceste modele la intrebuintari pentru cautare prin limbaj natural, iar in final integrandu-se cu mediile de dezvoltare software si cu terminalul sistemului de operare pentru a avea acces la un context mai bun dar si pentru executia de comenzi, nu a fost una dificila, avand in vedere competentele avansate de prelucrare si generare de text ale acestor modele.

Momentan, in domeniul modelelor de limbaj, exista cativa jucatori mari precum Google, Anthropic si OpenAI care controleaza o buna parte din cota de piata. Acestia sunt in stadiul in care isi fortifica ecosistemul pentru a-si maximiza retentia de utilizatori. Fiecare furnizor de astfel de servicii lanseaza produse care se folosesc de aceste modele. De la asistenti de cercetare, la asistenti de programare si la programe ce automatizeaza sarcini zilnice. Odata cu aceste produse, incearca sa restrictioneze sau sa ofere tarife diferite pentru utilizatori ce folosesc modelele pentru astfel de sarcini in afara ecosistemului lor.

Exista si modele de limbaj de tipul Open Source Software (OSS), de diferite marimi si cu diverse proprietati, dar ecosistemul uneltelor gratuite sau OSS care folosesc aceste modele pentru sarcini avansate inca este in proces de dezvoltare. Astfel, este un mediu propice pentru un produs ce livreaza un asistent inteligent pentru dezvoltarea software, independent de furnizor, construit pentru costuri reduse, performanta ridicata, abordarea problemelor complexe ce poate rula local, precum cel pe care imi propun sa il realizez in aceasta lucrare.

SCOPUL SI OBIECTIVELE LUCRARI

Scopul acestei lucrari este de a realiza un asistent inteligent extensibil si transparent bazat pe IA, agnostic fata de platforma sau de furnizori de modele de limbaj, ce ruleaza local sau in cloud si ce poate interactiona cu mediul virtual in care ruleaza (sistem de operare, mediu de dezvoltare, internet) prin intermediul uneltelor (tool-uri ¹), realizat de un workflow agentic ²

Obiectivul principal care trebuie atins este realizarea unui produs functional ce poate ajuta un inginer software in cat mai multe aspecte ale meseriei sale. Pentru a indeplini acest obiectiv, trebuie sa avem in vedere urmatoarele necesitati: interfata facila ce poate fi folosita oriunde folosind terminalul sistemului de operare (Terminal User Interface (TUI)), suficiente unelte pentru a indeplini sarcini cat mai complexe, management-ul conversatiei, contextului si al memoriei, versionare, evaluari si teste de regresie, monitorizare si observabilitate (metrici), siguranta (medii sigure pentru executarea codului, masuri de protectie pentru date de intrare malitioase, Out of Scope (OOS) detection), si cel mai important: un prompt de sistem ales cu grija.

¹In IA folosita cu ajutorul agentilor **tool**-urile sunt functii cu descrieri detaliate de folosire pe care agentul le populeaza cu parametrii, le executa si receptioneaza raspunsul pentru a-l folosi mai departe in rezolvarea problemelor complexe

²**Agentii** sunt aplicatii ce folosesc inteligenta artificiala pentru a interactiona cu mediul inconjurator

Capitolul 1

LUCRARI ANTERIOARE

O buna intelegere a tehnologiilor, standardelor si a obstacolelor actuale din industrie necesita studierea literaturii de specialitate, a lucrarilor si a arhitecturilor existente deja in domeniu. Acest pas este obligatoriu pentru alegeri informate in cadrul implementarii proiectului, evitarea problemelor deja recunoscute si confera o vedere de ansamblu asupra stadiului final al lucrarii.

1.1 Drumul catre modelele mari de limbaj (LLM)

Domeniul inteligentei artificiale a fost studiat o perioada indelungata iar cateva publicatii au marcat industria prin concepte revolutionare si progrese importante.

Unul din conceptele de baza ale inteligentei artificiale a aparut pentru prima data publicat in anul 1958 de catre F. Rosenblatt in lucrarea [15], unde este prezentat perceptronul: un model inspirat din fizionomia neuronului biologic, capabil sa realizeze sarcini de clasificare binara. Perceptronul primeste un set de date de intrare, fiecare asociata unei ponderi, iar rezultatul final este determinat prin combinarea acestor valori si aplicarea unei functii de activare. Procesul de invatare are loc prin ajustarea iterativa a ponderilor in functie de eroarea obtinuta la iesire, mecanism ce reprezinta baza invatarii automate moderne. Aceasta abordare a demonstrat pentru prima data posibilitatea construirii unor sisteme capabile sa invete din experienta, fara a fi programate explicit pentru fiecare situatie.

Cu toate acestea, desi perceptronul a reprezentat un pas revolutonar in dezvoltarea inteligentei artificiale, modelul prezenta limitari importante. In forma sa initiala, acesta putea clasifica doar date liniar separabile, ceea ce restrictiona capacitatea de rezolvare a problemelor complexe. Din acest motiv, perceptronul simplu era eficient doar pentru reprezentari de date de intrare relativ simple, intr-un singur strat de procesare. Aceste limitari au dus ulterior la dezvoltarea retelelor neuronale multistrat si a algoritmilor mai avansati de antrenare, care au stat la baza progreselor moderne din domeniul inteligentei artificiale.

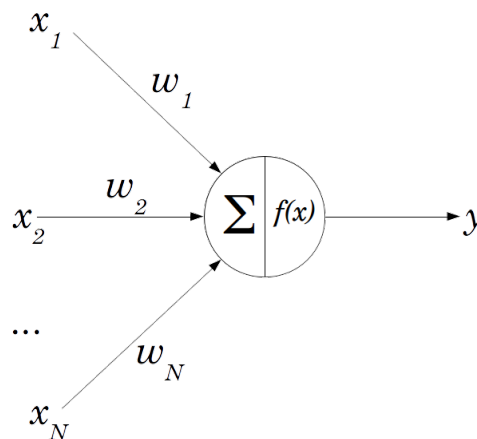


Figura 1.1: Modelul unui perceptron

Un progres major s-a ivit odata cu introducerea retelelor neurale convolutionale (Convolutional Neural Network (CNN)) de catre Yann LeCun si colaboratorii sai in lucrarea [16]. Aceasta abordare a fost conceputa pentru a depasi limitarile modelelor anterioare, precum perceptronul simplu care nu putea procesa eficient informatii complexe din imagini. Aparitia retelelor convolutionale a marcat inceputul aplicarii pe scara larga a inteligentei artificiale in domeniul procesarii si analizei imaginilor.

Ideea fundamentala din spatele acestor retele era relativ rudimentara, insa extrem de eficienta: datele de intrare nu mai sunt tratate ca pe un vector unidimensional, astfel modelul analizeaza portiuni locale ale imaginii prin intermediul unor "ferestre" de convolutie. Aceste filtre parcurg imaginea si extrag automat trasaturi relevante, precum muchii, forme sau texturi. In plus, utilizarea mai multor straturi succesive permitea retelei sa invete reprezentari din ce in ce mai abstracte si mai complexe ale datelor vizuale.

Un alt element important introdus de retelele convolutionale a fost utilizarea operatiilor specializate, precum convolutia si subsampling-ul (pooling), care reduceau dimensionalitatea datelor si imbunatateau capacitatea de generalizare a modelului. Astfel, CNN au devenit capabile sa identifice tipare spatiale complexe si sa obtina performante superioare in sarcini precum recunoasterea faciale, clasificarea imaginilor sau detectia obiectelor. Aceste inovatii au reprezentat fundamentul dezvoltarii ulterioare a tehnologiilor moderne de computer vision si deep learning.

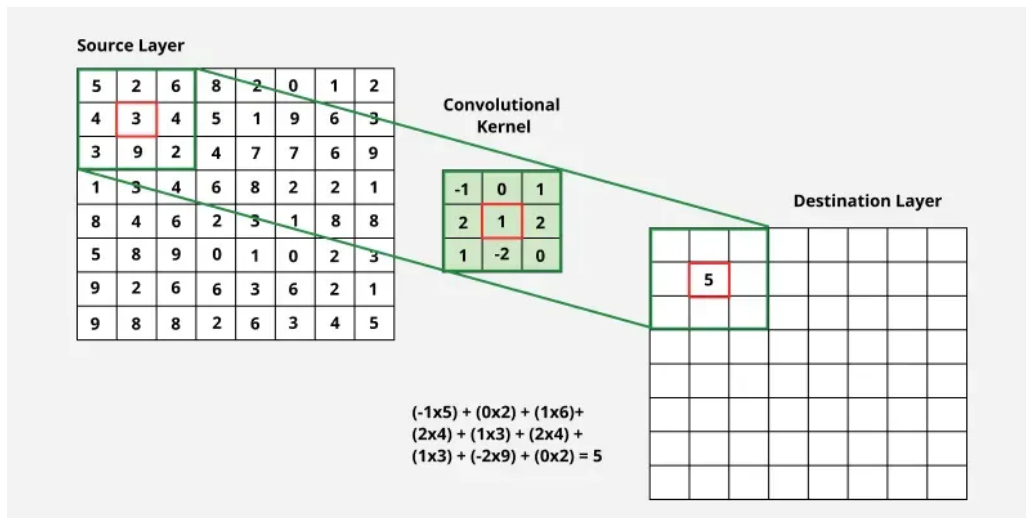


Figura 1.2: Convolutia cu un nucleu a unei imagini

Desi rețelele neuronale convolutive (CNN) au demonstrat performante remarcabile in procesarea datelor statice, precum imaginile, acestea nu erau potrivite pentru aplicatii in care informatia are caracter secvential. Domenii precum procesarea limbajului natural (Natural Language Processing (NLP)), recunoasterea vorbirii sau analiza datelor in timp presupun existenta unei dependente intre elementele succesive ale datelor, informatia desfasurandu-se in timp si nu intr-o structura fixa bidimensionala. Din acest motiv, utilizarea unor "ferestre" locale nu era suficienta pentru captarea relatiilor pe termen lung dintre elementele unei secvente.

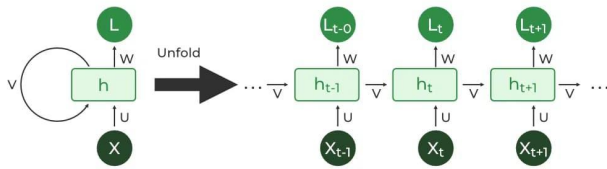
Primele incercari de rezolvare a acestei probleme au fost realizate prin intermediul rețelor neuronale recurente (Recurrent Neural Network (RNN)), arhitecturi capabile sa pastreze informatia din pasii anteriori prin intermediul unei stari interne. Astfel, modelul putea utiliza contextul anterior pentru a influenta predictiile curente, ceea ce reprezenta un progres important pentru procesarea datelor temporale. Cu toate acestea, antrenarea RNN clasice s-a confruntat rapid cu o problema fundamentala a invatarii profunde: disparitia gradientului.

Aceasta problema aparea in timpul procesului de propagare inversa prin timp (Backpropagation Through Time (BTT)), atunci cand secventele analizate deveneau foarte lungi. Gradientii calculati in timpul antrenarii se diminuau exponential pe masura ce informatia era propagata catre inceputul secventei, devenind atat de mici incat actualizarile ponderilor ajungeau neglijabile. In consecinta, rețeaua "uita" informatiile importante del a inceputul secventei si nu putea invata dependente pe termen lung.

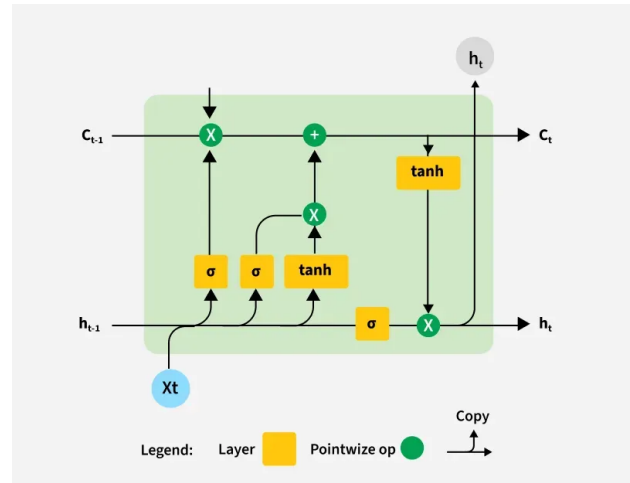
Un progress major a fost realizat de Sepp Hochreiter si Jürgen Schmidhuber prin introducerea arhitecturii Long Short Term Memory (LSTM) in lucrarea [17]. Acest model a oferit rețelor neuronale o forma functionala de memorie pe termen lung, capabila sa pastreze informatii relevante pentru perioade extinse de timp. Elementul central al acestei arhitecturi il reprezinta mecanismul de "porti" care controleaza fluxul informatiei in interiorul rețelei. Prin aceste mecanisme, modelul poate decide ce informatii trebuie pastrate, ce informatii trebuie actualizate si ce informatii trebuie eliminate din memorie.

Datorita acestei capacitati de gestionare eficienta a contextului temporal, rețelele LSTM au devenit fundamentul multor aplicatii moderne de inteligenta artificiala, fiind utilizate cu succes in traducere automata, recunoastere vocala, generare de text si analiza de secvente temporale, desi problema disparitiei gradientilor nu disparuse complet.

Punctul de cotitura in evolutia inteligentei artificiale a fost reprezentat de introducerea arhitecturii "transformer", prezentata de Vaswani si colaboratorii sai in lucrarea devenita celebra [18]. Pana in acel moment, modelele bazate pe LSTM dominau procesarea limbajului natural, insa acestea prezentau o limitare importanta: datele secventiale trebuiau procesate pas cu pas,



(a) RNN



(b) LSTM

Figura 1.3: RNN si LSTM comparate

fiecare element depinzand de rezultatul precedent. Aceasta caracteristica facea antrenarea pe volume foarte mari de date sa fie lenta si dificil de paralelizat, limitand scalabilitatea modelelor.

Arhitectura transformer a schimbat fundamental aceasta abordare prin introducerea mecanismului de "self-attention". In loc sa proceseze secvential, modelul putea analiza simultan toate elementele unei propozitii si putea calcula relatiile dintre acestea indiferent de distanta lor intext. Astfel, fiecare cuvnt primea informatii contextuale din intreaga secventa, ceea ce permitea captarea mult mai eficienta a dependentelor semantice si sintactice. In plus, eliminarea procesarii strict secventiale a permis paralelizarea operatiilor pe hardware modern precum Graphics Processing Unit (GPU), reducand semnificativ timpul de antrenare.

Mai tarziu, mecanismul de "self-attention" s-a transformat in "multi-head attention" care avea cate un cap de atentie pentru caracteristici specifice, putand astfel sa invete multiple tipuri de dependente intre date, fiecare cap specializandu-se pe un anumit tip mai mult sau mai putin abstract de dependenta intre date. Pe langa deblocarea intelegerii relatiilor complexe, Multi-Head Attention (MHA) a adus si o optimizare in timpul de antrenare si de inferenta, dar si la imbunatatirea generalizarii modelelor.

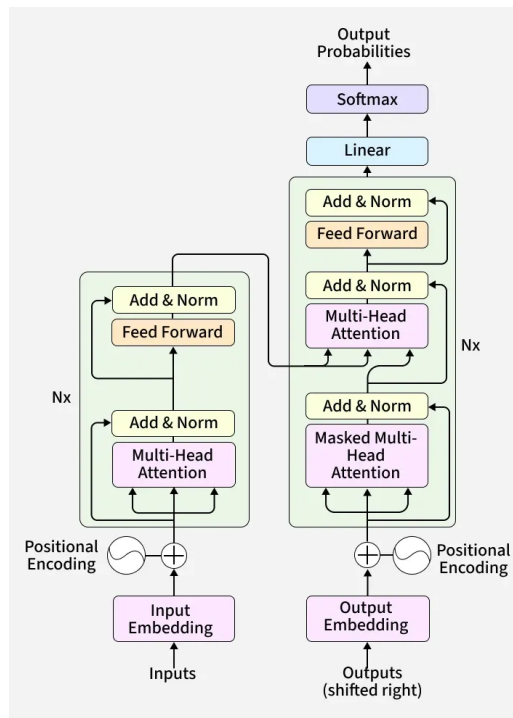


Figura 1.4: Arhitectura transformer

Odata eliminata limitarea din punct de vedere computational, cercetatorii au putut antrena modele pe seturi de date de dimensiuni fara precedent. Acest progres a deschis drumul catre aparitia modelelor mari de limbaj, LLM. Acestea sunt capabile sa invete reprezentari complexe ale limbajului natural prin analiza unor cantitati uriase de text provenite din carti, articole, pagini web si alte surse disponibile pe internet. Transformerul a devenit rapid fundamentul majoritatii sistemelor moderne de inteligenta artificiala dedicate limbajului natural, influentand profund atat cercetarea academica, cat si aplicatiile comerciale.

Modelele mari de limbaj reprezinta rezultatul direct al antrenarii retelelor pe volume masive de date si folosind un numar foarte mare de parametrii. Cercetatorii au observat ca performanta modelelor continua sa creasca odata cu extinderea dimensiunii acestora si a cantitatii de date utilizate la antrenare. Pornind de la aceasta idee, echipa OpenAI a dezvoltat modelul GPT-3, care includea aproximativ 175 de miliarde de parametrii, descoperirile fiind prezentate in lucrarea „Language Models are Few-Shot Learners” [19]. La momentul aparitiei, GPT-3 reprezenta unul dintre cele mai mari si mai performante modele lingvistice create.

Una din cele mai importante realizari ale acestui model a fost capacitatea de a rezolva sarcini complexe fara a necesita o antrenare suplimentara specifica fiecarei probleme. Prin simpla furnizare a catorva exemple sau chiar doar a unei instructiuni formulate in limbaj natural, modelul putea genera cod sursa, realiza sumarizari de text, raspunde la intrebari, traduce texte sau crea continut creativ precum poezii si eseuri. Aceasta particularitate a fost documentata articolul prezentat anterior a demonstrat ca modelele de mari dimensiuni pot generaliza cunostiinte intre domenii diferite si isi pot adapta comportamentul fara modificari suplimentare ale parametrilor interni.

Prin aceste progrese, inteligenta artificiala a facut tranzitia de la sisteme specializate, concepute pentru rezolvarea unei sarcini predefinite, la modele capabile sa manifeste competente generale avansate. Aceasta schimbare a reprezentat inceputul unei noi etape in dezvoltarea inteligentei artificiale moderne, in care accentul s-a mutat de la proiectarea unor solutii dedicate catre construirea unor modele inteligente universal, capabile sa interactioneze natural cu oamenii si sa rezolve probleme complexe.

1.2 Maturizarea modelelor mari de limbaj si cercetari actuale

Odata cu aparitia primelor foundation models ¹ precum GPT-4, GPT-5, Sonnet sau Opus, industria inteligentei artificiale a cunoscut o crestere exponentiala. Aceste modele prezinta o eficacitate impresionanta in rezolvarea problemelor din domenii diverse, de la generare si analiza textului, pana la programare, traduceri automate, automatizari complexe sau gandire logica. Datorita acestei flexibilitati, integrarea acestor modele a devenit o prioritate pentru majoritatea companiilor ce activau in domeniul tehnologiei.

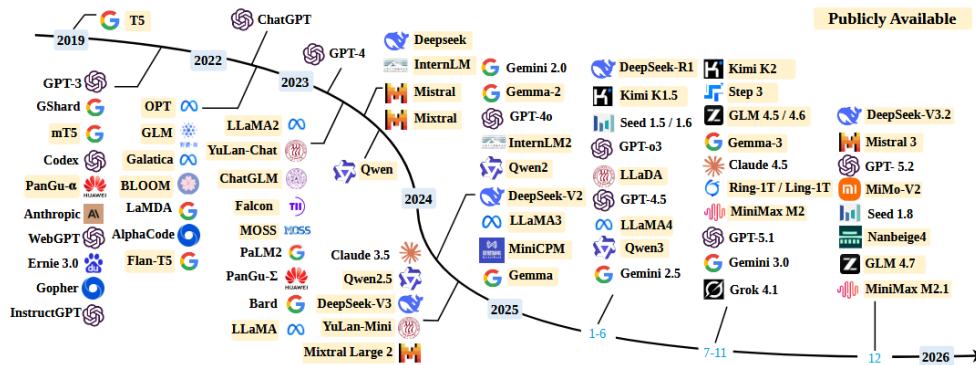


Figura 1.5: Evolutia recenta a modelelor mari de limbaj [1]

Astfel, cresterea rapida a cererii pentru sisteme bazate pe inteligenta artificiala a determinat furnizorii de astfel de modele si alte companii care ar beneficia de pe urma modelelor, sa investeasca masiv in cercetare si infrastructura. Aceste investitii au permis in final antrenarea unor modele cu capacitati din ce in ce mai sofisticate, fiecare noua generatie de modele aducand imbunatatiri semnificative in ceea ce priveste rationamentul, intelegerea limbajului si capacitatea de a rezolva probleme complexe.

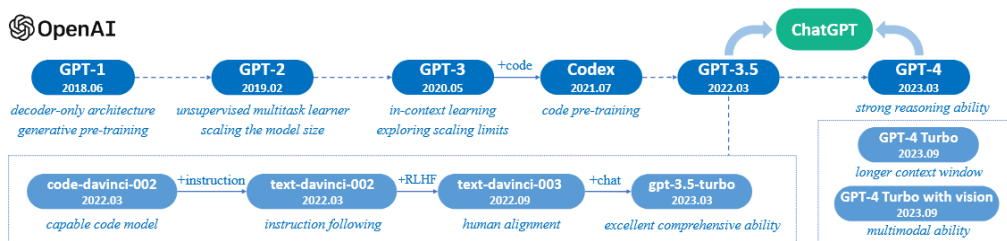


Figura 1.6: Evolutia modelelor GPT [1]

Investitiile in infrastructura nu au fost singurul motiv pentru care modelele de limbaj au avut o crestere impresionanta. Cercetarea a jucat de asemenea un rol crucial in optimizarea lor. Multe optimizari au fost aduse acestor modele de limbaj, printre care si imbunatatiri ale arhitecturii transformer care isi propun optimizeze modelele pentru procesarea secventelor mai lungi de date de intrare, precum este ilustrat in lucrarea "Longformer: The Long-Document Transformer" [2] de Beltagy si colaboratorii sai care isi propun sa optimizeze mecanismul de atentie, folosind serii trigonometrice pentru a stabili o compresie a cache-ului Key-Value (KV)

¹Un foundation model este un model mare de limbaj de uz general antrenat pe cantitati foarte mari de date diverse, ce poate fi adaptat pentru un numar mare de intrebuintari

² ³ imbunatatind astfel gandirea pe termen lung, lucru documentat in lucrarea "TriAttention: Efficient Long Reasoning with Trigonometric KV Compression" de Mao si colaboratorii sai.

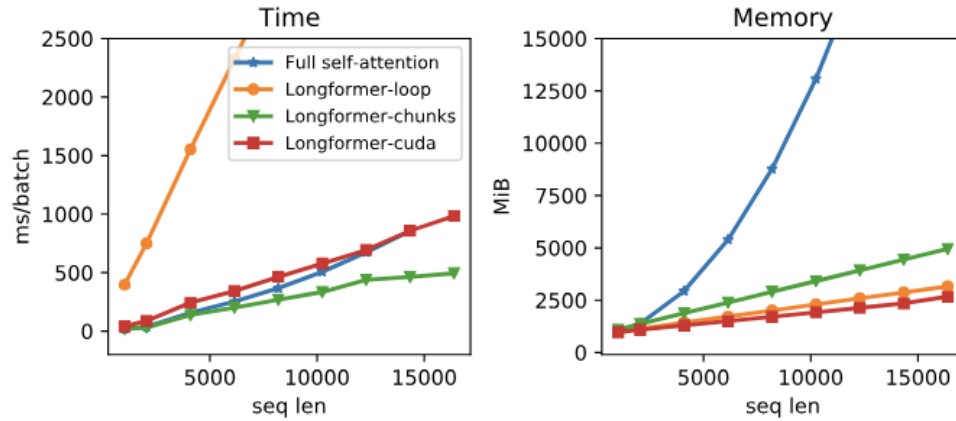


Figura 1.7: Performanta tipurilor de arhitecturi de tipul "Longformer" [2]. In timp ce mecanismul tipic de atentie utilizeaza memorie exponential, arhitectura Longformer foloseste memorie liniar odata cu cresterea contextului

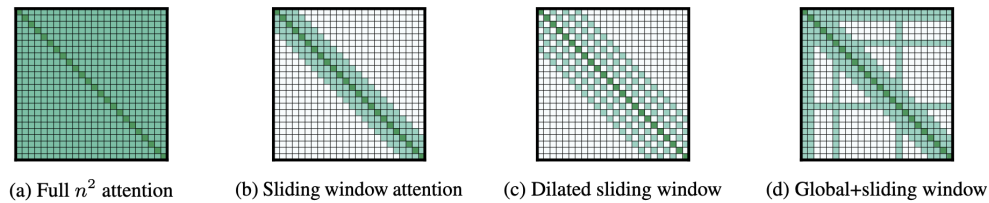


Figura 1.8: Diferite configuratii de atentie folosite in arhitectura Longformer in comparatie cu mecanismul obisnuit de atentie [2]

Inca un punct de optimizare al arhitecturii curente este imbunatatirea embedding-urilor, ⁴ precum studiul realizat in lucrarea "RoFormer: Enhanced Transformer with Rotary Position Embedding" [3] care isi propune sa optimizeze pozitia exacta a token-ului in secventa cu o matrice de rotatie in timp ce incorporeaza dependenta pozitiei relative in formula mecanismului de atentie, reusind astfel sa capteze informatii importante din secventa de intrare precum, lungimea secventei, relatii intre tokens care se diminueaza pe masura ce distanta relativa dintre ei creste, echipand astfel mecanismul de atentie cu o codare pozitionala relativa.

²Un token reprezinta datele de intrare pentru arhitectura transformer si uzual sunt cuvinte, parti din cuvinte sau caractere. Acestea permit modelului sa sparga un cuvant posibil mai rar in parti mai cunoscute.

³Cache-ul KV este o optimizare fundamentala a arhitecturii transformer ce elimina calculele redundante ale token-urilor anterioare in generarea autoregresiva, conform lucrarii "KV Cache Optimization Strategies for Scalable and Efficient LLM Inference" de Xu si colaboratorii [20]

⁴Embedding-ul este transformarea token-ului intr-un vector abstract multi-dimensional ce reprezinta informatia extrasa de model din acel token, plus un vector ce aduce informatii despre pozitia acestui in secventa.

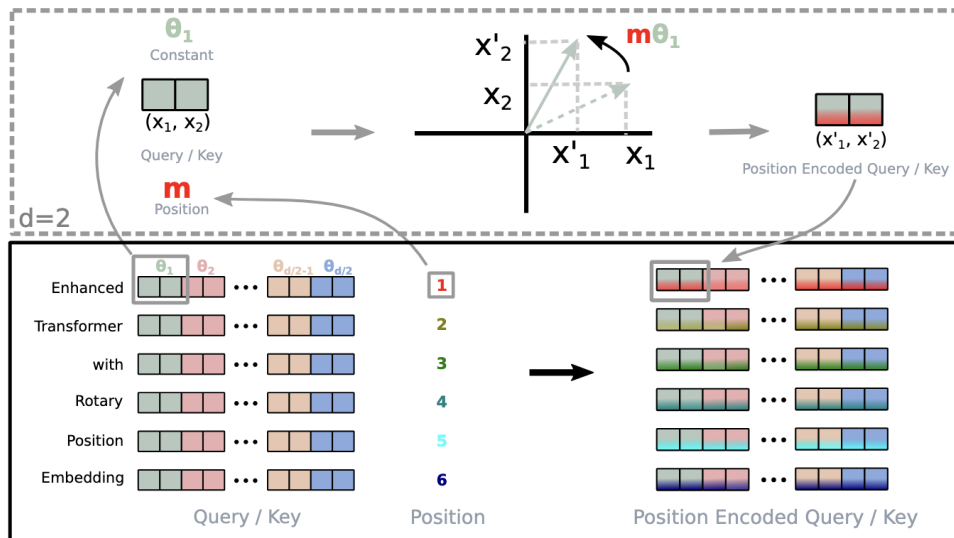


Figura 1.9: Principiul de baza pentru Rotary Position Embedding [3]

Imbunatatiri s-au adus si in partea de fine-tuning⁵ mai ales prin lucrarea LoRA: Low-Rank Adaptation of Large Language Models" [4] care aduce imbunatatiri prin inghetarea ponderilor modelului si injectarea unor matrici antrenabile de descompunere in fiecare strat al arhitecturii transformer, reducand numarul de parametrii antrenabili, reducand numarul acestor de pana la 10.000 de ori si puterea de calcul necesara de 3 ori in comparatie cu fine-tuning-ul normal.

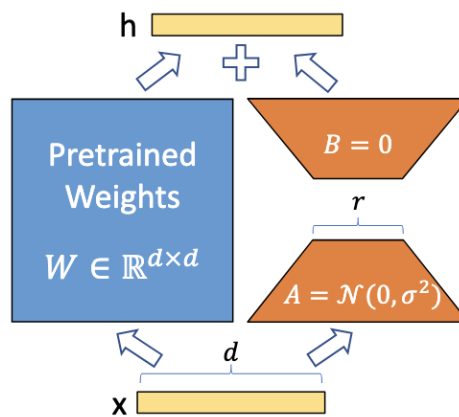


Figura 1.10: Fine-tuning-ul se realizeaza doar pe A si B [4]

Performanta a vazut de asemenea progrese, spre exemplu prin procedeul de decodare speculativa prezentata in articolul "Fast Inference from Transformers via Speculative Decoding" [21] care ilustreaza problema timpului de inferenta⁶ care continua sa creasca pe masura ce marimea modelelor creste si ea. Autorii propun un algoritm care sa permita livrarea mai multor tokeni in paralel. La baza lucrarii stau doua observatii fundamentale: (1) majoritatea sarcinilor abordate de modelele mari de limbaj pot fi scindate in obiective mai usoare care pot fi abordate

⁵Fine tuning-ul este un procedeu prin care parametrii unui model sunt ajustati pentru eficienta sporita intr-un anumit domeniu.

⁶Inferenta este procedeul prin care un model de limbaj ia datele de intrare, aplica transformarile necesare si produce un raspuns final.

de modele mai eficiente si (2) folosind executia speculativa si o noua metoda de esantionare, decodarea poate fi accelerata rulandu-le in paralel pe baza iesirilor modelele mai eficiente ce aproximeaza pe cat de bine pot, generand mai multi tokeni simultan fara a modifica distributia.

Continuand incercarile de a eficientiza modelele mari de limbaj, a aparut intrebarea: "Daca marimea modelul incetinesc antrenarea si inferenta, de ce nu cautam sa micșorăm modelul?". Astfel, au aparut procedeele de cuantizare ⁷ si distilare ⁸. Problema distilării este nevoia unui model superior care sa poarte rolul de "invatator". Zhang si echipa sa de cercetatori documenteaza o alternativa a acestei metode prin procedeul de distilare proprie in lucrarea "Embarrassingly Simple Self-Distillation Improves Code Generation" [5] prin esantionarea cu anumite valori ale temperaturii ⁹ si configuratii ale trunchierii, in final executand fine-tuning supervizat ¹⁰ pe acele esantioane.

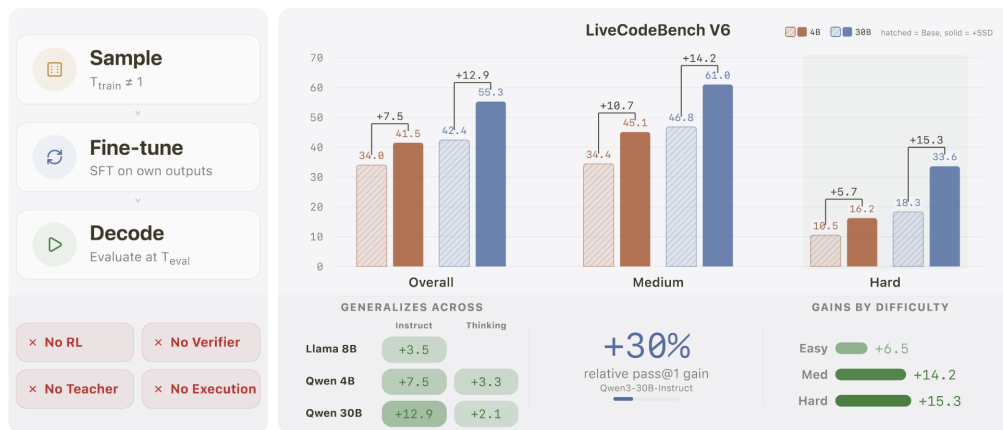


Figura 1.11: Rezultatele procesului de Simple Self Distillation [5]

⁷Cuantizarea este reducerea tipului de date care stocheaza parametrii rețelei. Spre exemplu daca in loc de 8 biti, stocam parametrii pe 4 biti, injumatatim amprenta modelului

⁸Distilarea este antrenarea unui model mai mic prin obiectivul reproducerii raspunsurilor modelului mai mare

⁹Temperatura este o setare pentru modelele de limbaj care controleaza "creativitatea" modelului. O temperatura mica (spre exemplu 0) obliga modelul sa scoata la iesirea sa cel mai probabil token posibil, pe cand o temperatura mare (spre exemplu 0.9), permite modelului sa aleaga un alt token probabil.

¹⁰Supervizarea se refera la abilitatea modelului de a folosi exemple etichetate corect de oameni in prealabil pentru antrenare. Antrenarea supervizata nu include raspunsurile corecte, iar modelul trebuie sa descopere singur tiparele ce reprezinta raspunsul corect

LiveCodeBench v6 02/2025 - 05/2025									LiveCodeBench v5 08/2024 - 02/2025										
	PASS@1				PASS@5					PASS@1				PASS@5					
	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD		ALL	EASY	MED	HARD	ALL	EASY	MED	HARD		
THINKING																			
Qwen3-4B-Thinking	54.5	97.4	59.2	29.7	67.5	100.0	75.4	45.8	59.6	98.0	69.5	31.4	70.3	98.9	80.1	47.5			
+ SSD	57.8 +3.3	98.7 +1.3	62.8 +3.6	33.8 +4.1	71.4 +3.9	100.0	77.3 +1.9	53.1 +7.3	63.1 +3.5	98.5 +0.5	74.1 +4.6	35.7 +4.3	74.7 +4.4	98.9	86.8 +6.7	52.8 +5.3			
Qwen3-30B-Thinking	66.1	99.8	73.0	41.5	76.7	100.0	84.7	59.7	69.1	98.6	81.7	43.9	79.8	98.9	91.4	61.1			
+ SSD	68.2 +2.1	100.0 +0.2	76.4 +3.4	46.7 +5.2	80.2 +3.5	100.0	87.1 +2.4	65.8 +6.1	69.9 +0.8	98.4 -0.2	82.9 +1.2	44.9 +1.0	80.5 +0.7	98.9	91.8 +0.4	62.4 +1.3			
INSTRUCT																			
Llama-3.1-8B-Instruct	12.7	46.8	5.4	0.0	23.0	76.7	15.1	0.8	12.7	45.1	4.1	0.7	20.8	64.7	11.6	2.8			
+ SSD	16.2 +3.5	52.6 +5.8	10.0 +4.6	1.6 +1.6	24.0 +1.0	73.3 -3.4	17.1 +2.0	3.3 +2.5	13.5 +0.8	45.2 +0.1	5.9 +1.8	1.3 +0.6	21.9 +1.1	67.1 +2.4	12.3 +0.7	3.7 +0.9			
Qwen3-4B-Instruct	34.0	79.7	34.4	10.5	41.0	90.0	40.5	16.5	34.3	85.4	28.4	10.1	45.4	96.0	45.4	17.5			
+ SSD	41.5 +7.5	86.5 +6.8	45.1 +10.7	16.2 +5.7	56.8 +15.8	98.1 +8.1	59.7 +19.2	34.1 +17.6	45.7 +11.4	98.2 +4.8	52.1 +23.7	16.6 +6.5	61.9 +16.5	98.8 +2.8	73.1 +27.7	33.6 +16.1			
Qwen3-30B-Instruct	42.4	84.5	46.8	18.3	53.5	93.3	56.8	31.1	45.8	91.2	49.8	17.7	58.7	96.1	69.2	38.6			
+ SSD	55.3 +12.9	91.0 +6.5	61.0 +14.2	33.6 +15.3	71.6 +18.1	99.9 +6.6	76.4 +19.6	54.1 +23.0	54.3 +8.5	92.9 +1.7	64.1 +14.3	25.9 +8.2	70.7 +12.0	97.6 +1.5	83.0 +13.8	47.2 +16.6			

Figura 1.12: Imbunatatiri in sarcini de programare ale modelelor dupa ce au fost supuse procesului de Simple Self Distillation[5]

Pe de alta parte, problema cuantizarii este dezbatuta in lucrarea "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate" [22] ce isi propune sa cuantizeze vectori multi-dimensional fara distorsiuni majore ale datelor. Acest lucru este posibil prin transformari ale vectorilor de intrare, precum rotatii aleatoare inducand o distributie "Beta" si folosindu-se de independenta aproximativa ale coordonatelor distincte in vectorii multi-dimensional. Se reuseste astfel minimizarea memoriei necesare, a timpului de inferente si a latimii de banda necesare pentru rularea modelului prin reducerea preciziei numerice. Acesta este eficient si in cazul ponderilor modelului, dar este foarte important si in cache-ul KV deoarece permite rularea modelelor cu ferestre de context ¹¹ lungi, optimizand necesitatile de memorie. Odata cu aceasta descoperire, au venit adaptari ale procedeele anterioare, precum LoRA [4] in cazul modelelor cuantizate. O astfel de lucrare este "QLoRA: Efficient Finetuning of Quantized LLMs" [6] care isi propune sa imbunatateasca fine-tuning-ul pentru modele cuantizate.

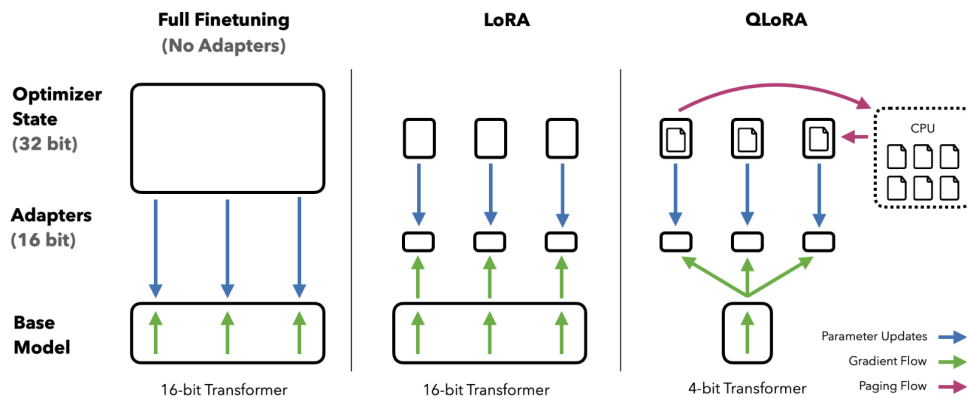


Figura 1.13: Comparatia procedeele de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [6]

Pe langa optimizarile aduse arhitecturii transformer, o buna parte din evolutia gandirii adanci se bazeaza pe procedeul de Reinforcement Learning (RL). Pe scurt, modelul este recompensat mai mult sau mai putin in functie de raspunsurile pe care le da, iar obiectivul sau

¹¹Contextul se refera la

este sa fie recompensat cat mai mult. Raspunsurile care prezinta gandire, mai exact Chain-of-Thought (CoT) sunt recompensate mai mult decat cele ce nu prezinta aceasta particularitate, iar raspunsurile gresite, care nu sunt fundamentate sau contin fraze nedorite sunt penalizate. In felul acesta, modelele invata sa prefere sa scoata date de iesire care prezinta particularitati ale gandirii, care duc in final la raspunsuri mai logice si capabilitatea de a rezolva probleme mai complexe, avand in vedere functionarea decodurului din transformer care genereaza pas cu pas, in functie de pasii anteriori. In lucrarea "HARDTESTGEN: A High-Quality RL Verifier Generation Pipeline for LLM Algorithmic Coding" [23], de exemplu e discutata importanta RL si a modalitatii de a avea semnale mai bune de recompensa pentru aplicatii de programare prin generarea de cazuri de test complexe, pe care verificatorul le evalueaza si recompenseaza modelul in functie de raspunsul dat.

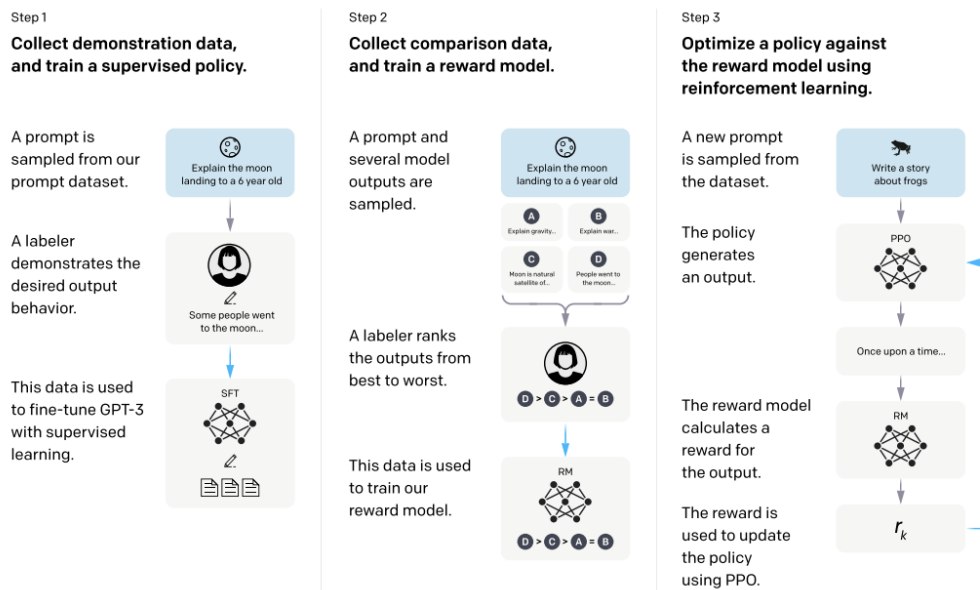


Figura 1.14: Ilustrarea procedeului de RL [7]

In final, o alta latura care a beneficiat de atentie sporita este prompt-ul ¹² pe care il primeste modelul. Modelele de limbaj, fiind autoregresive adica generand date de iesire pe baza datelor anterioare sunt dependente de un prompt bine gandit care maximizeaza contextul pe care il are modelul si care ii impune limite si directive pe care sa le respecte. Importanta acestui prompt fiind accentuata si de o noua arie de specializare, de "Prompt Engineering" ce presupune constructia unor prompt-uri eficiente, lucru ilustrat in lucrarea "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques" [24].

Avand in vedere nevoia de prompt-uri ce produc rezultate consistente si versionabile, echipa condusa de Khattab a publicat lucrarea "DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines" [25], ce propune o abordare sistematica pentru a crea si optimiza prompt-uri. Au introdus astfel un model de programare ce abstractizeaza aceste prompt-uri prin grafuri de transformari textuale sau grafuri computationale imperative si un compilator care functioneaza cu acest nou limbaj, rezultatele fiind instructiuni optimizate ce intrec performantele prompt-urilor scrise de mana.

¹²Prompt-ul este reprezentat de datele de intrare ale modelelor de limbaj, instructiunile pentru care trebuie sa produca un raspuns la iesire

1.3 Solutii agentice pentru industria dezvoltarii software

Arhitecturile agentice presupun inzestrarea modelelor mari de limbaj cu uneltele necesare interactionarii cu ecosistemul in care ruleaza. Fundamentul acestor arhitecturi a fost pus de procedeul CoT [8], precedat mai tarziu de Tree-of-Thought (ToT) [9] care a demonstrat ca un model de limbaj poate naviga probleme complexe prin descompunerea rationamentului in pasi intermediari expliciti. Aceasta tehnica a deschis calea catre generarea de planuri structurate, iar Wang si colaboratorii sai au extins aceasta idee prin metoda Plan-and-Solve [26], in care modelul este instruit sa conceapa mai intai un plan de actiune ce divide sarcina in sub-obiective mai mici, iar apoi sa le execute conform planului stabilit. Abordarea a reusit sa elimine erorile de pasi lipsa frecvente in CoT clasic, obtinand performante comparabile cu promptarea few-shot pe probleme de rationament matematic, fara a necesita exemple manuale.

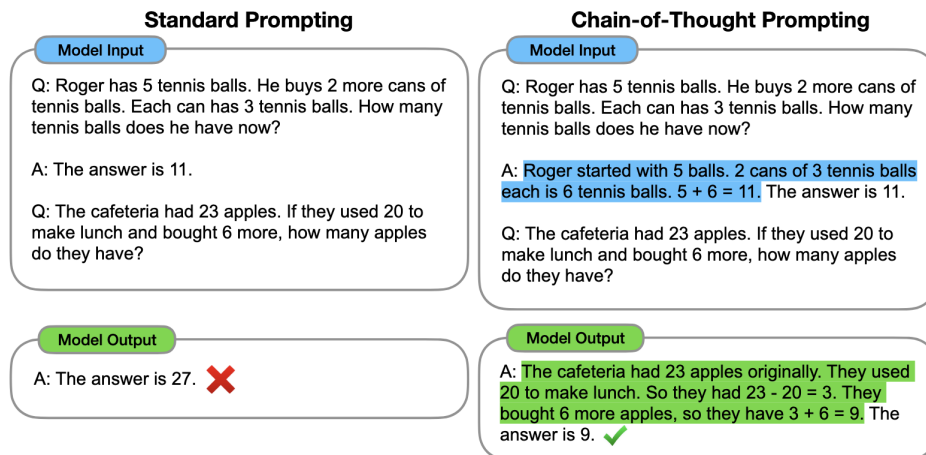


Figura 1.15: Exemple de CoT [8]

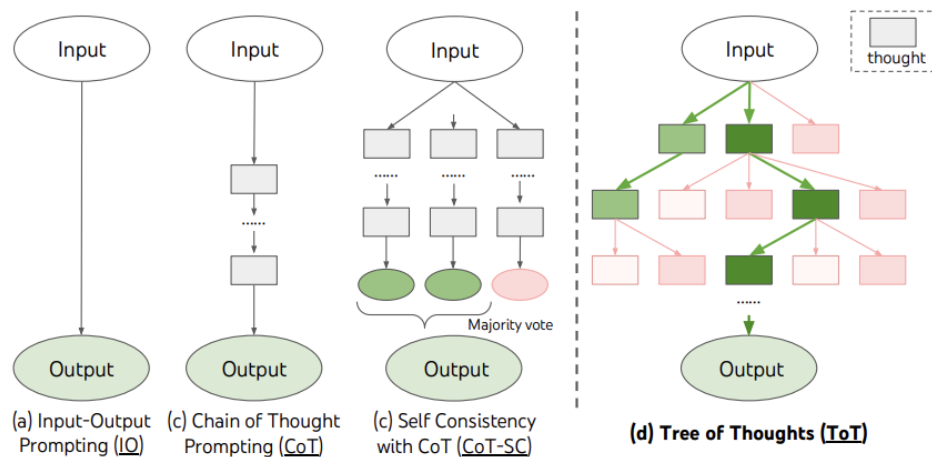


Figura 1.16: Diferenta intre procedee de gandire [9]

Mai tarziu, abordarea ReAct [10] a combinat rationamentul cu actiunea, permitand modelului sa alterneze intre generarea de ganduri intermediare si executia de actiuni concrete in mediul extern. Aceasta sinergie intre gandire si actiune a incercat sa combata fenomenul halu-

cinatiilor ¹³ si inconsistenta in raspunsuri cu care se confruntau primele modele mari de limbaj. Totusi, paradigma ReAct presupunea o intercalare costisitoare intre rationament si observatii: modelul genera un gand, apela o unealta, astepta raspunsul, apoi decidea urmatoarea actiune pe baza tuturor token-urilor anterioare.

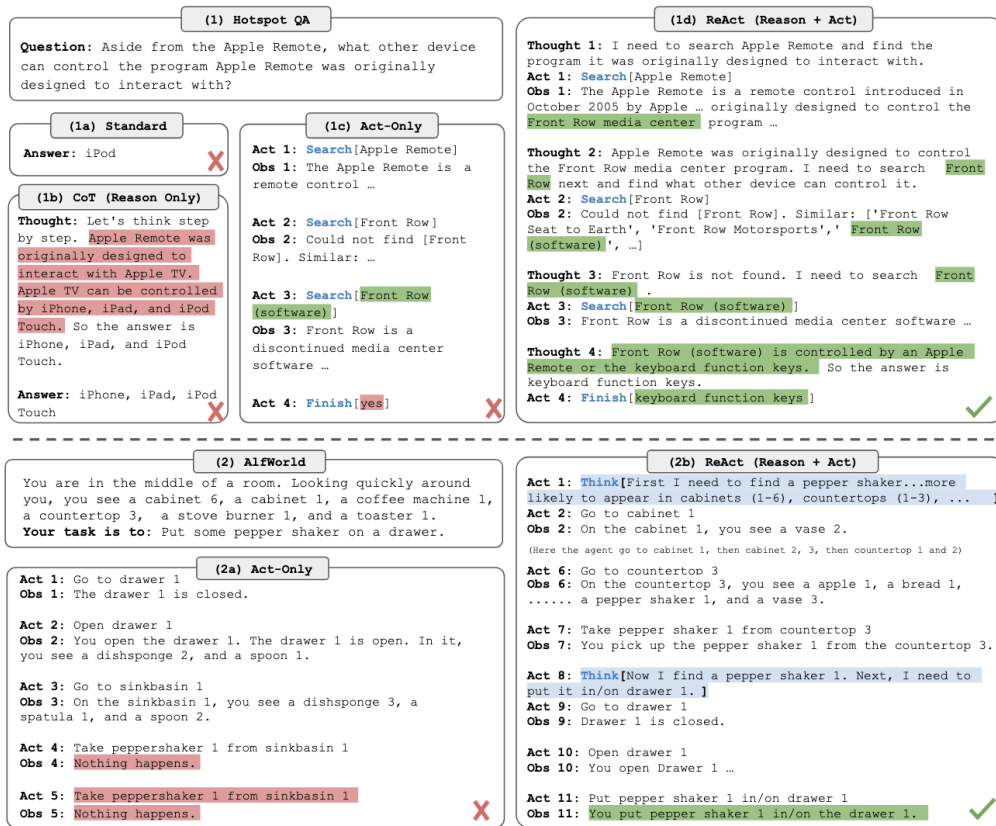


Figura 1.17: Paradigma ReAct [10]

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Figura 1.18: Imbunatatirile pe care le aduce ReAct [10]

Xu si colaboratorii sai au identificat aceasta ineficienta si au propus arhitectura ReWOO [11] care decupleaza complet procesul de rationament de observatiile externe. In loc sa astepte raspunsul fiecarei unelte inainte de a planifica urmatorul pas, modelul genereaza intregul plan de actiuni dintr-o singura trecere, iar observatiile sunt colectate independent si integrate ulterior. Aceasta separare a redus consumul de tokeni de pana la 5 ori si a imbunatatit acuratetea cu 4% pe benchmark-ul HotpotQA, demonstrand totodata robustete crescuta in scenarii de esec al uneltelor.

¹³Fenomenul de halucinatii in modelele de limbaj se refera la generarea unor raspunsuri care nu sunt ancorate in fapte reale sau care nu au legatura cu prompt-ul de intrare

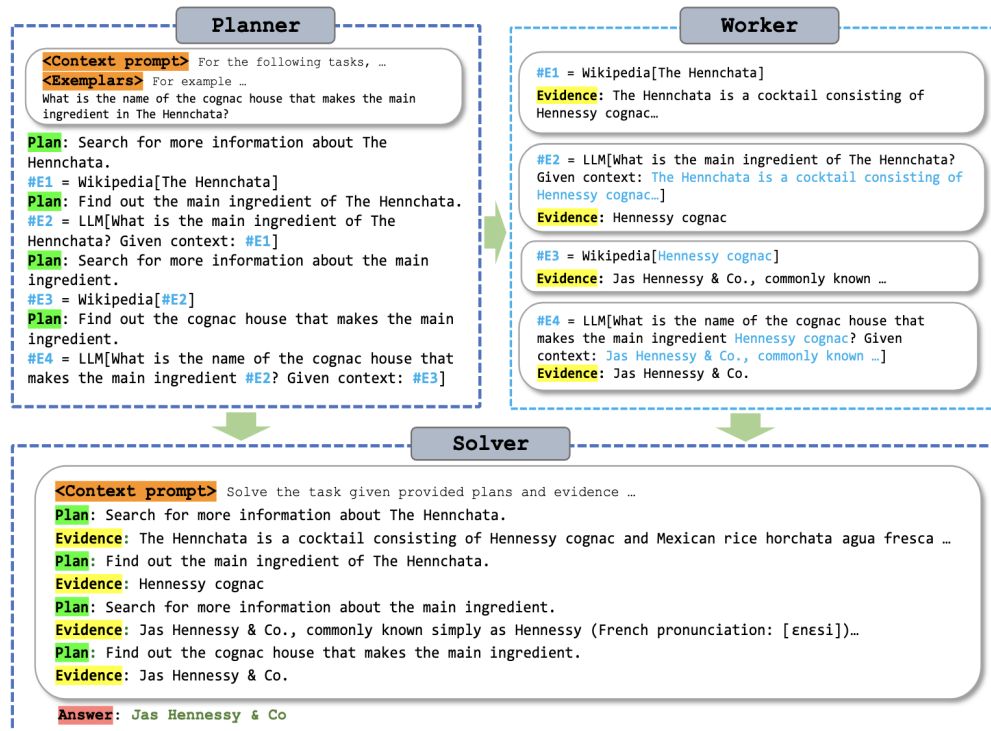


Figura 1.19: Ideea din spatele ReWOO [11]

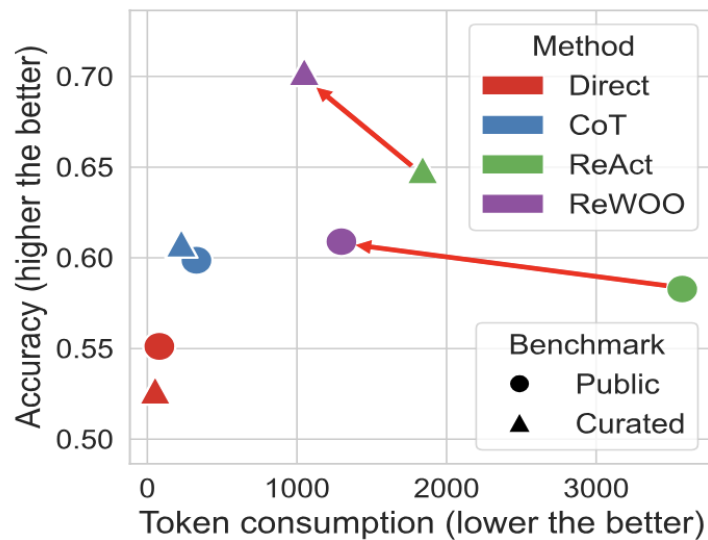


Figura 1.20: Comparatia de performante intre paradigme pentru imbunatatirea gandirii din punctul de vedere al acuratetii vs. numarul de tokeni consumati [11]

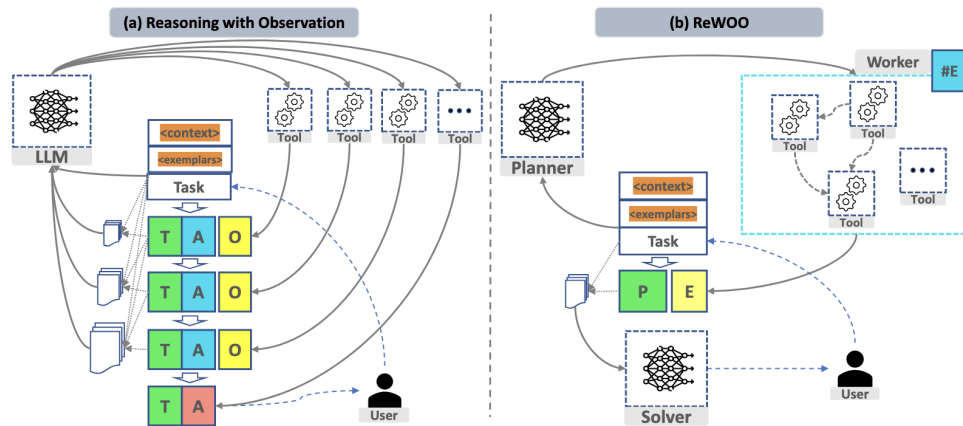


Figura 1.21: Imbunatatirea adusa de ReWOO este delegarea mai multor libertati catre worker ce poate apela mai multe unelte pana sa produca un raspuns. P = Plan, E = Evidence, A = Answer, O = Observation, T = Thought [11]

Increderea in procesul de gandire al modelelor de inteligenta artificiala s-a cimentat pe masura ce aceste tehnici au inceput sa produca rezultate consistente, iar astfel a urmat inezestrarea acestora cu unelte care sa le permita interactiunea cu mediul in care ruleaza. Numarul uneltelor ce le ofera capacitatea de a cauta pe internet pentru a gasi date actuale si a nu se limita la datele de antrenare, la posibilitatea rularii de comenzi in terminalul sistemului de operare sau chiar preluarea controlului browserului personal pentru automatizari, a crescut exponential, si odata cu el si numarul de aplicatii ale acestor modele.

Cu toate acestea, pe masura ce agentii au inceput sa proceseze cantitati din ce in ce mai mari de informatii din mediul in care opereaza, fereastra de context a devenit o limitare critica. Desi, multe modele sunt promovate ca avand o fereastra de context maxima de 1 milion de tokeni precum Opus 4.7 de la Anthropic sau GPT-5.5 de la OpenAI, modelele se lovesc de anumite limitari in a accesa tot acel context. Liu si colaboratorii sai au demonstrat in lucrarea "Lost in the Middle" [12] ca performanta modelelor de limbaj se degradeaza semnificativ atunci cand informatia relevanta se afla in mijlocul unui context lung. Modelele tind sa utilizeze eficient informatiile de la inceputul si de la sfarsitul secventei de intrare, dar pierd din vedere elementele plasate in zona centrala, chiar si in cazul modelelor concepute explicit pentru contexte lungi. Aceasta descoperire are implicatii directe asupra agentilor care acumuleaza istoricul conversatiei, apelurile la unelte si observatiile intr-un context tot mai voluminos.

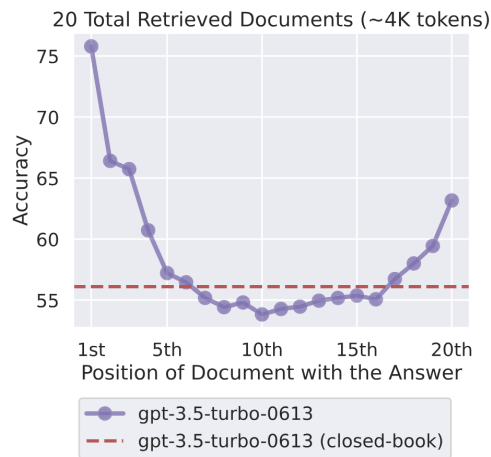


Figura 1.22: Modele tind sa aiba o acuratete foarte slaba contra intrebarilor ce vizeaza informatii care se afla la mijlocul ferestrei curente de context [12]

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1] (Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ . Subrahmanyan Chandrasekhar shared...

Document [2] (Title: List of Nobel laureates in Physics) **The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...**

Document [3] (Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen

Figura 1.23: Vizualizarea problemei si al rezultatului dorit la intrebare. De multe ori, modelele intampina dificultati in accesarea acestor informatii ascunse in mijlocul ferestrei de context [12]

Pentru a evalua riguros aceste limitari, Vodrahalli si echipa sa au propus framework-ul Michelangelo [13], o metoda de evaluare sintetica si minimala care depaseste testele clasice de tip 'needle-in-a-haystack'. In loc sa masoare doar capacitatea de a regasi o singura informatie dintr-un context lung, Michelangelo testeaza abilitatea modelului de a sintetiza si corecta informatii dispersate, relevand o structura latentă din datele de intrare. Rezultatele au confirmat ca exista un spatiu semnificativ de imbunatatire in procesarea informatiei din contexte lungi, chiar si pentru modelele de ultima generatie.

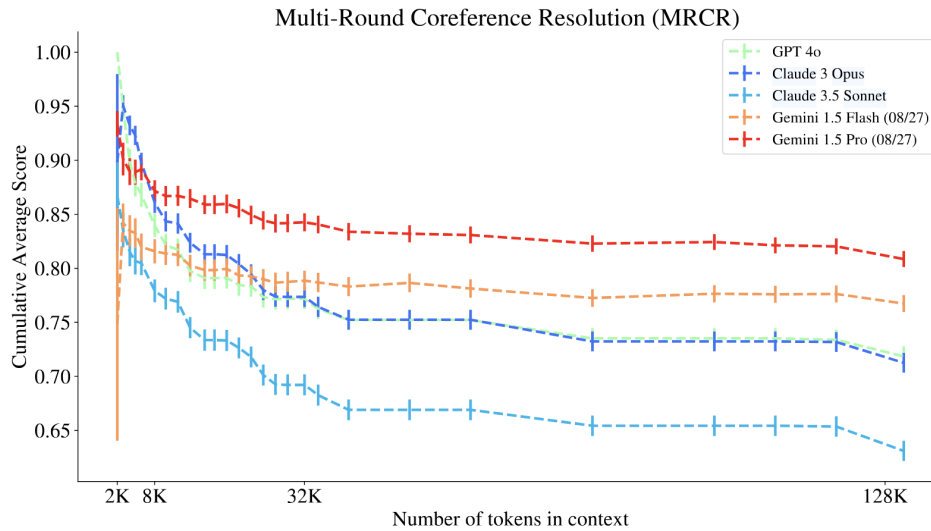


Figura 1.24: In majoritatea modelelor se observa o scadere a performantelor dupa 32K tokeni pe sarcini de gandire complexa[13]

Din punct de vedere al eficientei computationale, Li si colaboratorii sai au propus Lychee-Cluster [27], o metoda de gestionare a cache-ului KV care pastreaza coerenta semantica locala prin fragmentare bazata pe delimitatori si construiesc un index ierarhic recursiv. Aceasta abordare transforma cautarea in cache dintr-o scanare liniara intr-un proces de eliminare in timp logarithmic, obtinand o accelerare de pana la 3.6 ori a inferentei cu o degradare neglijabila a performantelor modelului. Pentru aplicatiile agentice care implica conversatii lungi cu ape-uri frecvente la unelte, astfel de optimizari sunt esentiale pentru a mentine latentia scazuta si costurile sub control.

Industria dezvoltarii software are probabil una din cele mai profitabile intrebuintari ale acestor arhitecturi agentice, datorita volumului masiv de munca existent in domeniu, asa ca nu a durat mult pana ce marii jucatori pe piata modelelor de IA au inceput sa ofere produse care sa asiste programatorii in sarcinile lor. La momentul actual exista mai multe solutii competitive: Codex de la OpenAI, Gemini CLI de la Google sau Claude Code de la Anthropic. Toate acestea sunt solutii proprietare care sunt facute sa ruleze cu modelele proprii si pentru care utilizatorul trebuie sa plateasca. Exista si solutii open-source precum OpenCode, Kiro sau OpenClaw cu care detinatorii de modele se lupta pentru a domina piata, desi lupta nu este stransa din cauza monopolului solutiilor proprietare.

Companie	Model	Pret [\$/MTok]	Capabilitati
Anthropic (link)	Claude Opus 4.7	\$5/input MTok, \$25/output MTok	Cel mai capabil model disponibil, imbunatatire semnificativa pentru programare agentica fata de 4.6, gandire adaptiva, 1M fereastră de context, 128k tokens max. la iesire
	Claude Opus 4.6	\$5/input MTok, \$25/output MTok	Lent, cel mai inteligent, gandire adaptiva, 1M fereastră de context, 128k tokens max. la iesire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niste inteligenta pentru performante, gandire adaptiva, 1M fereastră de context, 64k tokens max. la iesire
	Claude Haiku	\$1/input MTok, \$5/output MTok	Cel mai rapid, fara gandire adaptiva, 200k fereastră de context, 64k tokens max. la iesire
OpenAI (link)	gpt-5.5	\$5/input MTok, \$30/output MTok	Noua clasa de inteligenta pentru programare si munca profesionala, gandire adaptiva cu niveluri multiple de efort (low/medium/high/xhigh), suport multimodal, 1M fereastră de context si 128k tokens max. la iesire
	gpt-5.4	\$2.5/input MTok, \$15/output MTok	Cel mai inteligent pentru sarcini agentice, de programare si profesionale. 1M fereastră de context si 128k tokens max. la iesire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare si subagenti. 400k fereastră de context si 128k tokens max. la iesire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic si foarte rapid pentru sarcini usoare in volum mare. 200k fereastră de context si 128k tokens max. la iesire
Google (link)	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok >= 200k tokens	Model pentru sarcini complexe ce necesita cunostiinte dezvoltate. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligenta comparabila cu modelul Pro la o viteza mai mare. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse si volum mare de sarcini. Fereastră de context 1M in / 64k out
	Gemma 4	Open Source	Familie de modele open-source pentru rulare locala, cost redus si personalizare
DeepSeek (link)	DeepSeek V4	\$0.30/input MTok, \$0.50/output MTok	Model general de ultima generatie, suporta mod cu si fara gandire adanca, 1M fereastră de context, cache hit: \$0.03/input MTok
	DeepSeek R1	\$0.55/input MTok, \$2.19/output MTok	Model specializat in rationament si gandire adanca pentru probleme complexe, 128k fereastră de context, cache hit: \$0.14/input MTok
Meta (link)	Llama 4	Open Source	Performante similare cu GPT4-o sau Gemini 2.0 Flash, dar open-source. Fereastră de context 10M/1M in functie de model.
Qwen/Alibaba (link)	Qwen 3	Open Source	Familie de modele open-source (Apache 2.0) cu arhitectura hibrida MoE, gandire adaptiva activabila, performante frontier la programare si rationament. Disponibil in variante de 0.6B pana la 235B parametri
	Qwen 3.6 Plus	\$0.325/input MTok, \$1.95/output MTok	Varianta API proprietara (Alibaba Cloud), arhitectura hibrida MoE cu atentie liniara, gandire adaptiva, 1M fereastră de context, 64k tokens max. la iesire
Mistral AI (link)	Mistral Large 3	\$0.50/input MTok, \$1.50/output MTok	Cel mai capabil model Mistral, open-weight, multimodal, performante de nivel frontier. 262k fereastră de context
	Mixtral 8x22B	Open Source	Model Mixture-of-Experts open-source (Apache 2.0), 8 experti de 22B parametri fiecare, disponibil pentru rulare locala sau prin API. 66k fereastră de context

Tabela 1.1: Comparatia catorva din modelele existente si a capabilitatile acestora in comparatie cu pretul de utilizare

Nota: Toate modelele au si versiuni pentru diferite nivele de efort pentru gandire (reasoning effort) sau pentru date multimodale ce pot influenta si costul dar si rezultatele

Din tabelul 1.1 se poate deduce ca utilizarea frecventa a unui model de ultima generatie poate ajunge cu usurinta sa coste o mica avere. Cu o mica aproximare, 200.000 de tokens sunt echivalentul unei carti de 400 de pagini. Dar contorizarea nu este facuta doar pentru instructiunile originale. Intr-o conversatie cu un model de limbaj fiecare pereche raspund-intrebare din istoricul conversatiei respective este introdus ca date de intrare pentru urmatorul prompt pe care utilizatorul il introduce. Astfel, in loc de cativa tokeni de intrare, intr-o conversatie lunga cu un istoric dezvoltat, costul de intretinere al conversatiei creste exponential.

De asemenea, fiecare apel catre unelte externe precum cautarile web, raman in istoricul conversatiei pentru ca modelul sa aiba context in query-urile ulterioare. Fiecare dintre aceste apeluri la unelte consuma o buna parte din fereastra de context a modelului, crescand costul.

Din punct de vedere al capabilitatii de rezolvare a sarcinilor reale din industria dezvoltarii software, clasamentul celor mai bune modele se schimba frecvent. Utilizand SWE-Bench Pro [14], putem masura puterea de abordare a acestor modele de top contra unor sarcini complexe.

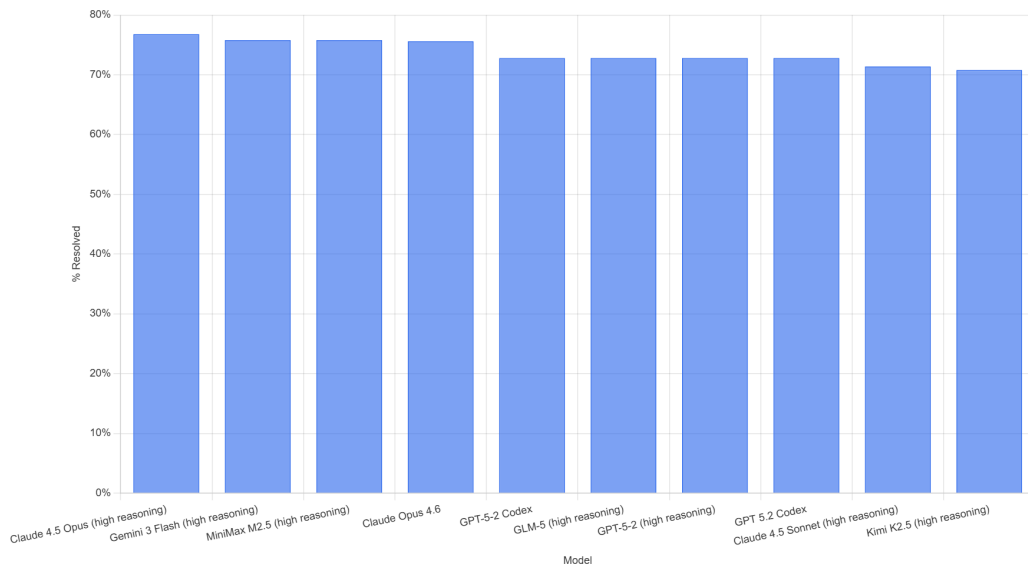


Figura 1.25: Clasamentul modelelor in SWE-Bench Pro [14] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza

Pe langa puterea de rezolvare, un alt fenomen a aparut care schimba rata de adoptie a unui model: costul. Desi performantele modelelor cresc si acestea sunt din ce in ce mai integrate in vietile noastre de zi cu zi, asemenea telefoanelor mobile, costul de rulare al acestor modele creste si el. Cele mai puternice modele au nevoie de foarte multe resurse pentru antrenare dar si pentru inferenta. Acest lucru se reflecta asupra utilizatorului final care trebuie sa plateasca per-token sau un abonament lunar care include un numar limitat de tokeni. Modelele cu capacitati de gandire adanca, pe langa faptul ca au un cost mai mare per-token, consuma si mai multi tokeni din cauza gandirii dezvoltate sau a ferestrei de context mari, ridicand si mai mult costurile. Pentru asta, fiecare detinator de modele fundamentale, are mai multe versiuni ale aceluiasi model, pentru intrebuintari de diverse complexitati, pentru a ajuta cu costurile dar si cu balansarea resurselor. Spre exemplu Google detine modelele din gama Gemini, cu versiunile "Flash", "Thinking", "Pro".

Avand in vedere problema costului dar si a balansarii traficului, aplicatiile de asistenti de programare de pe piata, au implementat o modalitate de a clasifica prompt-urile utilizatorului in functie de complexitate si rutarea acestui prompt catre un model potrivit sa rezolve aceasta problema si optim din punct de vedere al costului.

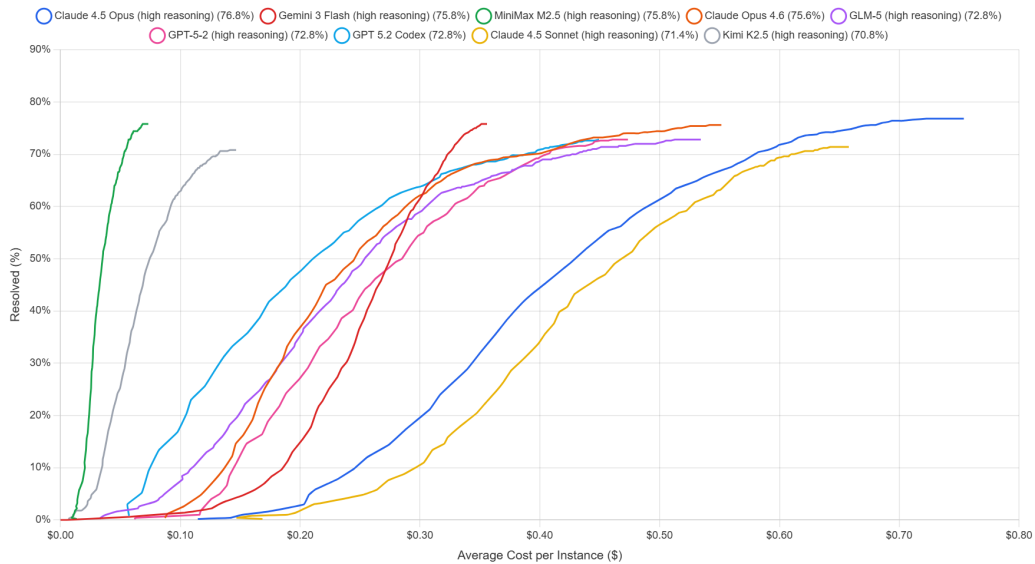


Figura 1.26: Clasamentul modelelor in SWE-Bench Pro [14] la momentul actual in functie de cost

1.4 Necesitati pentru dezvoltarea unei solutii agentice in domeniul dezvoltarii software

Pentru a realiza un asistent inteligent in domeniul dezvoltarii software este nevoie sa atingem cateva obiective principale, printre care: o interfata accesibila din diferite medii de dezvoltare pentru a satisface cat mai multe nevoie, unelte ce pot rula in terminalul oricarui sistem de operare, posibilitatea de a alege dintr-o sumedenie de modele, comenzi pentru a controla parametrii modelelor si un prompt de sistem si skill-uri specializate pentru operatii din domeniu.

Agentii functioneaza, pe langa procedeele expuse anterior, pe una dintre aceste filozofii de programare: Spec-Driven Development (SDD) sau Test-Driven Development (TDD). Primul se refera la programarea pe baza unor documente de specificatie ce expun in detaliu toate necesitatile implementarii, iar cel din urma se refera la programarea dupa cerintele sumare si executarea testelor la final pentru a confirma comportamentul dorit.

Pe langa aceste obiective de baza, trebuie sa ne asiguram ca modelul raspunde doar prompt-urilor cu scop legitim, si sa ne asiguram ca modelul nu poate rula comenzi care pun in pericol integritatea sistemului pe care ruleaza sau compromite date sensibile, lucruri pe care le vom discuta in capitolele ce urmeaza.

1.5 Puncte slabe ale arhitecturilor agentice

Desi entuziasmul pentru bula IA atinge cote maxime, inca este mult loc de imbunatarie, iar aceste platforme de asistenti de programare au inca multe defecte pe care trebuie sa le depasasca.

In primul rand, bariera principala in avansarea acestor asistenti este fereastra de context. Aceasta fereastra reprezinta cata informatie poate sa tina minte agentul. Pentru sisteme dezvoltate cu zeci de mii de linii de cod, sau solutii impartite pe micro servicii in mai multe proiecte, acest context limiteaza masiv capabilitatile agentilor. Exista multe tehnici de optimizare a acestui context de la compresie, realizarea unor fisiere ce rezuma cunostiintele actuale si inceperea unei conversatii noi care pleaca de la acel rezumat pana la rularea de cod pentru actiuni repetitive sau alegerea cu grija a prompt-ului. Dar din pacate toate aceste tehnici nu sunt de ajuns. Multe functionalitati care fac un agent suficient de performant, precum tool-urile

sau Retrieval Augmented Generation (RAG) [28] reusesc sa consume rapid aceasta fereastră de context. Poate 200.000 de tokeni pot parea un numar mare, dar defapt chiar si acest numar similar cu memoria hardware este defapt mai mic. Din aceasta fereastră doar 160.000 de tokeni sunt disponibili cu adevarat. O parte din ei sunt rezervati pentru auto-compactare sau pentru iesire. Depasirea acestei ferestre de context scade astfel performantele agentului si se recomanda inceperea unei conversatii noi.

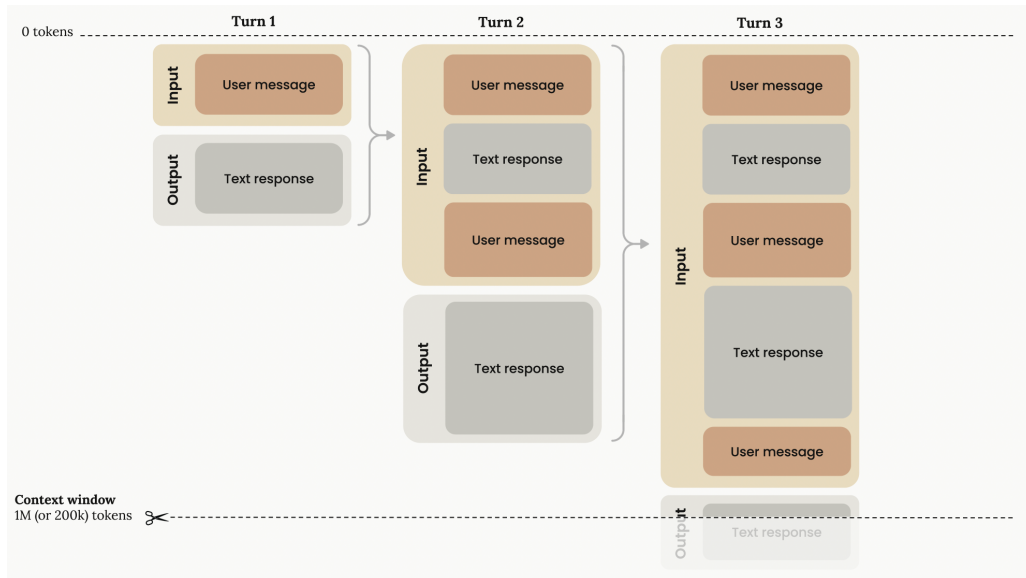


Figura 1.27: Reprezentarea ferestrei de context intr-o conversatie uzuala (sursa)

In al doilea rand, modelele curente par sa genereze date de iesire mai detaliate decat este necesar, preferand cantitatea peste calitate. De multe ori in sarcini de tipul "PR Review", agentii se grabesc sa relateze multe probleme simple sau de formatare si rateaza probleme importante de logica.

In al treilea rand, similar cu punctul anterior, agentii nu au o buna intelegere legata de cand ar trebui sa opreasca investigatiile sau sa incerce alte solutii si raman blocati in bucle infinite de a incerca aceleasi lucruri si a esua.

Pentru toate aceste probleme se lucreaza in mod curent la rezolvare de care vom vorbi ulterior.

1.6 Alegerea solutiei de implementare pe baza lucrarilor studiate si a standardelor din industrie

Solutia finala include mai multe categorii care trebuiesc comparate independent.

Pentru inceput, modelele disponibile initial vor fi din gama Gemini, Claude si OpenAI, cu posibilitatea de extindere la modele open-source rulate local sau in cloud.

Pentru partea de framework agentic, ofertele sunt numeroase si variate. Fiecare dintre optiunile de mai jos are avantaje si dezavantaje care trebuiesc cantarite in contextul aplicatiei noastre.

In materie de limbaj de programare, Python a fost alegerea evidenta deoarece este aproape folosit in totalitate in astfel de aplicatii, devenind un standard in industrie. Ecosistemul este de asemenea foarte matur si exista o sumedenie de biblioteci pentru a ne ajuta in impelmentare. De asemenea inca un atu este portabilitatea cu care vine, Python putand fi instalat cu usurinta pe orice sistem de operare cu o simpla comanda.

Am pornit de la ideea ca trebuie sa avem nevoie de un framework gandit pentru utilizat de unelte cu suport pentru mai multe modele, care sa suporte mai multe ramuri de executie, sa incorporeze managementul starii si al memoriei si sa poata construi un workflow robust si consistent. Pentru asta, am adunat o colectie de framework-uri folosite in industrie pentru a le compara si a decide care este cel potrivit.

Tabela 1.2: Comparație framework-uri pentru workflow-uri agentice

Criteriu	LangChain	LangGraph	Google Gen AI / Vertex	LiteLLM	OpenAI SDK
Rol Principal	Orchestrator general de LLM-uri și integrări.	Framework pentru agenți stateful și fluxuri ciclice.	Platformă enterprise integrată în Google Cloud.	Proxy sau router universal pentru 100+ LLM-uri.	Client oficial pentru modelele OpenAI.
Nivel Abstr.	Înalt (multe componente gata de utilizat).	Mediu/Înalt (control fin asupra grafurilor).	Înalt (Managed Services în Vertex AI).	Jos (strat de traducere API).	Jos/Mediu (interacțiune directă).
Capabilități Agentice	Limitate (predominant fluxuri liniare).	Excelent (bucle, feedback și multi-agent).	Bun (suport nativ pentru tool-uri).	N/A (doar rutare de mesaje).	Depinde de utilizator (manual sau Assistants API).
Suport Multi-Model	Da (agnostic).	Da (prin ecosistemul LangChain).	Optimizat pentru Gemini.	Absolut (scopul principal).	Nu (doar OpenAI).
Management Stare	Încorporat (Memory buffers).	Superior (persistență și „time travel”).	Gestionat de platformă.	N/A (stateless).	Threads (doar în Assistants API).
Învățare	Medie (ecosistem foarte mare).	Abruptă (necesită logica grafurilor).	Ușoară (UI intuitiv în Cloud).	Ușoară (identic cu OpenAI SDK).	Foarte Ușoară, documentat

În final am ales să folosesc LangGraph datorită controlului mare pe care îl oferă asupra desfășurării workflow-ului agentice și a integrării cu LangSmith ¹⁴ dar și pentru compatibilitatea cu logica desfășurării procesului de asistență în dezvoltarea software, respectiv buclele de scriere de cod, feedback, investigare și îmbunătățire, în ciuda complexității mai mari pe care o constituie acest framework.

¹⁴Unealta de observabilitate open-source pentru agenți

Pentru interacțiunea cu utilizatorul, ideea de pornire a fost ca trebuie să avem o interfață care poate fi rulată în orice context. Gândul m-a purtat automat către un TUI deoarece orice sistem de operare detine un terminal care poate fi pornit în orice mediu de programare, oferind o gamă largă de utilizatori potențiali. Mai departe am constituit o listă de abordări și librării pe care le putem folosi pentru a construi această interfață, fiecare cu avantaje și dezavantaje care trebuie comparate.

Tabela 1.3: Comparatie biblioteci pentru interfața cu utilizatorul

Bibliotecă	Arhitectură	Avantaje	Dezavantaje
Textual	Motor TUI pe ecran complet folosind buffer-ul alternativ al terminalului. Arbore de widget-uri bazat pe DOM (similar cu HTML)	• Sistem de layout similar cu CSS (grid, dock) • Formatare de text încorporată nativ • Complet asincron și bazat pe evenimente	• Folosește un scrollbar fals (nu funcționează scroll-ul nativ) • Supraîncărcarea DOM-ului cauzează lag dacă mesajele vechi nu sunt șterse • Dificultate la copierea/lipirea nativă a textului
Prompt Toolkit și Rich	Experiență interactivă folosind buffer-ul principal al terminalului. Afișare prin stdout standard.	• Derulare perfectă prin scrollbar-ul nativ al terminalului • Zero lag sau probleme de memorie pe măsură ce istoricul crește • Selectarea și căutarea nativă a textului funcționează impecabil	• Nu este un motor real de layout UI (doar flux liniar) • Dificil de implementat panouri laterale sau complexe • Limitat strict la input ancorat în partea de jos
Urwid	TUI clasic pe ecran complet folosind buffer-ul alternativ al terminalului. Ierarhie strictă de widget-uri.	• Extrem de matur, stabil și testat în producție • Randare foarte performantă pentru liste mari • Gestionare excelentă a input-ului standard din terminal	• API învechit și curbă de învățare mai abruptă • Lipsește stilizarea modernă încorporată • Sincron implicit (necesită integrare suplimentară)

Initial, abordarea cu Textual parea corecta pentru ca ofera un sistem de layout foarte puternic care poate fi folosit pentru a construi o interfata complexa cu panouri laterale si meniuri. Insa, in timpul implementarii a trebuit sa abandonez aceasta idee deoarece am intampinat probleme cu comportamentul interfetei in cadrul unei conversatii mai lungi cu elemente animate, pe care o sa le dezvoltam in capitolul de implementare.

Astfel, am finalizat lista alegerilor arhitecturale principale, vom motiva si restul alegerilor punctual, pe parcursul implementarii.

Capitolul 2

PLASAREA IN INDUSTRIE

In acest capitol vom aborda problematica plasarii proiectului in industrie si alegerile pe care le-am facut pentru a propune un produs competitiv.

2.1 Open source software

Momentan, cea mai mare cota de piata o au furnizorii de modele mari de limbaj precum Anthropic, Google sau OpenAI care lanseaza aplicatii care au ca obiectiv blocarea utilizatorilor in ecosistemul lor personal, prin unelte ce urmaresc domenii diverse.

In timp ce exista si alternative open source, acestea se lovesc de problemele tipice pentru astfel de programe. Pentru inceput, companiile ce furnizeaza accesul la aceste modele au tarife diferentiale pentru utilizarea cheilor Application Programming Interface (API) in astfel de programe si orice incercare de a insela sistemul poate rezulta in restrictionarea accesului la platformele furnizorilor.

De asemenea, o alta problema intampinata de platformele open source sunt lipsa fondurilor sau numarul mic de contribuitori, suportul pentru suita intreaga de functionalitati a programelor detinute de marile firme sau publicitate.

In ciuda acestor dificultati, avand in vedere cresterea vertiginioasa a costurilor, faptul ca accesul la program este gratuit este un mare plus pentru utilizatori. Un alt avantaj este suportul pentru modele de limbaj open source. Acestea sunt foarte usor de folosit chiar pe calculatorul utilizatorului daca detine componente hardware suficiente pentru modelul ales, reducand costurile si mai mult. Desi si pentru solutiile proprietare se pot face ajustari care sa permita folosirea modelelor open source, este preferat un program a carui scop este o platforma agnostica de furnizor.

2.2 Flux agentic ce ruleaza local

Un alt plus este confidentialitatea datelor si controlul asupra aplicatiei. Toate datele raman pe dispozitivul sau serverul utilizatorului (exceptand cazurile in care acesta alege explicit sa foloseasca modele de la furnizori si nu modele open source). Acesta ofera un mare avantaj pentru domenii in care se lucreaza cu date importante sau care necesita o confidentialitate ridicata.

Toate fisierele de configuratie sunt disponibile local si au o sumedenie de setari pe care utilizatorul le poate modifica dupa bunul plac.

Ruland aplicatia local, agentul are acces direct la mediul de lucru si astfel are context despre sarcinile pe care trebuie sa le execute. Mai mult, este eliminata necesitatea interventiei umane (pe langa actiunile periculoase ce necesita permisiuni speciale). Acesta poate executa actiuni si prelua controlul asupra dispozitivului, permitand astfel automatizari complexe, recurente sau care ruleaza o singura data, usurand sarcini repetitive salvand astfel foarte mult timp utilizatorului.

Actiunile pe care le poate rula un agent local sunt numeroase datorita integrarii cu terminalul sistemului de operare prin care poate controla o multime de aspecte.

2.3 Observability

Majoritatea panourilor cu informatii de utilizare de la furnizorii de modele de limbaj sunt sumare sau insuficiente. Aplicatiile in terminal au uzual mai multe date de utilizare spre deosebire de website-urile in care se desfasoara experienta conversationala, dar nu ofera prea multe date despre ce unelte in mod cert consuma cel mai mult context.

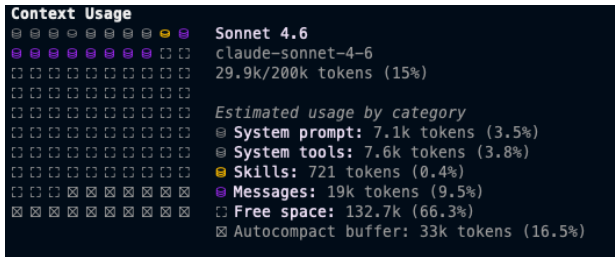


Figura 2.1: Reprezentarea ferestrei de context intr-o conversatie in Claude Code

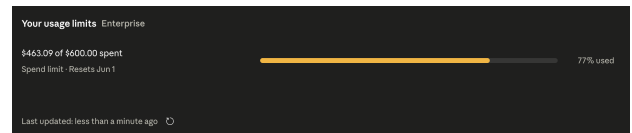


Figura 2.2: Reprezentarea utilizarii in Claude (varianta web) (sursa)

Astfel, in implementarea acestui asistent de programare, observabilitatea va juca un rol important pentru transparenta cu utilizatorul. Acesta poate interoga orice metrica disponibila in legatura cu consumul de tokeni si functionarea sistemului in timp real folosind Langfuse cat si panoul web inclus cu aplicatia.

2.4 Securitate si confidentialitate

Din punctul de vedere al datelor utilizatorului si confidentialitate utilizarea aplicatiei ofera confidentialitate totala. Datele acestuia nu pleaca dispozitivul sau decat daca acesta opteaza pentru modele ale furnizorilor sau daca isi configureaza Langfuse pentru observabilitate.

In ceea ce priveste securitatea si integritatea sistemului, programul va avea mai multe masuri, optionale sau obligatorii care se asigura ca agentul nu poate realiza actiuni cu totul distructive. Cateva din aceste sisteme sunt: rularea comenzilor intr-un container pe principiul de sandboxing, gardarea logica a apelarii uneltelor (protejarea apelarii secventiale a uneltelor, de exemplu trebuie sa citeasca un fisier inainte sa scrie in el), si gardarea intrarilor si iesirilor contra comportamentelor malitioase.

2.5 Cost

Costul este minim pentru asistentul de programare pe care urmeaza sa il prezint in urmatoarele capitole. Costul de utilizare este gratuit. De asemenea, daca utilizatorul alege sa opteze pentru modele open source, singurele costuri pe care va trebui sa le suporte pe langa achizitionarea unui dispozitiv capabil sa ruleze aceste modele sunt curentul electric si internetul.

Mai mult, aplicatia este optimizata pentru a reduce consumul de tokeni, deci si in cazul folosirii furnizorilor de modele, costul va fi unul pe cat se poate de redus, lasand la o parte taxele puse de acestia pe utilizarea modelelor in aplicatii terte.

2.6 Aplicatii rivale

Pe langa aplicatiile furnizorilor exista si cateva aplicatii open source care s-au bucurat de o oarecare adoptie.

Tabela 2.2: Top 6 asistenti de programare open source

Aplicatie	Audienta si scop	Puncte forte	Puncte slabe	Functionalitati cheie
OpenClaw (GitHub) MIT	Utilizatori care doresc un asistent AI personal, always-on, accesibil pe orice canal de comunicare	• Cel mai popular proiect open source din categorie • Suport multi-canal masiv (WhatsApp, Telegram, Slack, Discord, iMessage, Signal etc.) • Sandboxing cu Docker, SSH si OpenShell • Aplicatii native macOS, iOS, Android • Agnostic fata de furnizor, gateway local-first	• Orientat spre asistent personal, nu exclusiv programare • Configurare complexa pentru multi-canal • Necesita Node.js 22+ si infrastructura gateway • Nu are integrare directa in IDE	Gateway local, multi-canal, sandboxing, voice wake, live canvas, cron, skills, multi-agent, apps native
OpenHands (GitHub) MIT	Dezvoltatori si echipe enterprise care necesita un agent autonom complet cu izolare securizata	• Sandboxing in containere Docker – cel mai robust din lista • GUI web si CLI disponibile • SDK Python pentru constructia de agenti personalizati • Suport Kubernetes si enterprise • Agnostic fata de furnizor (Claude, GPT, Ollama etc.)	• Necesita Docker – cerinte de resurse ridicate • Configurare initiala complexa • Nu are integrare directa in IDE • Curba de invatare mai abrupta	Sandboxing, SDK, GUI web, CLI, Kubernetes, integrari Slack/Jira/Linear
Cline (GitHub) Apache 2.0	Dezvoltatori care prefera lucrul direct din IDE cu control granular si automatizari avansate	• Extensie VS Code si plugin JetBrains • SDK Node.js si CLI headless pentru CI/CD • Echipe multi-agent si agenti programati • Suport MCP si plugin-uri extensibile • Agnostic fata de furnizor (200+ modele via OpenRouter)	• Consum ridicat de tokeni datorita contextului mare • Fara sandboxing nativ incorporat • Complexitate ridicata – multe functionalitati • Dependent de ecosistemul VS Code/JetBrains	Extensie IDE, SDK, CLI, multi-agent, MCP, agenti programati, integrari Slack/Discord/Telegram
Goose (GitHub) Apache 2.0	Utilizatori generali si dezvoltatori care doresc un agent AI nativ, performant si extensibil	• Aplicatie nativa desktop (macOS, Linux, Windows) • Construit in Rust – performanta ridicata • Suport MCP (70+ extensii) • Agent general – nu doar pentru cod • Agnostic fata de furnizor (15+ furnizori, suport ACP)	• Mai putin specializat pe programare • Ecosistem de extensii mai tanar • Nu are integrare directa in IDE • Documentatie in curs de maturizare	App nativa, CLI, API, MCP, ACP, Rust, multi-platform, Linux Foundation
Aider (GitHub) Apache 2.0	Dezvoltatori individuali care prefera un workflow minimalist din terminal cu integrare Git	• Mapare automata a codului sursa (repo map) • Integrare Git nativa – commit-uri automate • Suport 100+ limbaje de programare • Linting si testare automata • Agnostic fata de furnizor (Claude, GPT, DeepSeek, Ollama)	• Fara sandboxing • Exclusiv terminal – fara GUI • Capabilitati agentice limitate fata de concurenta • Nu are extensie nativa de IDE	Terminal, Git auto-commit, repo map, voice-to-code, linting, modele locale

2.7 Mediul de dezvoltare

Expunerea elementelor constitutive disponibile care alcatuiesc mediul hardware cat si software este imperativa pentru relatarea procesului de dezvoltare si reproducerea pasilor prin care s-a ajuns la produsul final.

2.7.1 Cerinte hardware

Cerintele hardware pentru dezvoltarea si rularea acestei aplicatii sunt minime. In cazul in care folosim modelele din cloud este necesar doar un laptop sau computer ce poate rula Python 3 cu conexiune activa la internet. Deoarece inferenta nu este realizata local, nu este necesar un GPU performant sau o cantitate mare de RAM. Daca se doreste rularea unui model local, cerintele hardware depind de modelul ales, dar cresc exponential necesitand acces la o placa video moderna si o cantitate semnificativa de RAM.

2.7.2 Cerinte software

Cerintele software pentru rularea acestei aplicatii sunt minime de asemenea. Este necesar doar un sistem de operare cu terminal precum Windows, Linux sau macOS, cu Python 3 instalat si acces la internet.

In ceea ce priveste mediul de dezvoltare orice mediu de dezvoltare cu suport pentru Python este suficient, iar singurul program necesar este `uv`¹ pentru a instala dependintele necesare.

2.8 Structura

Structura aplicatiei reflecta separarea responsabilitatilor in mai multe directoare specializate. Directorul `src` contine logica agentului: graful `LangGraph`, uneltele, sistemul de permisiuni, clientul `Model Context Protocol (MCP)`, pipeline-ul de compactare, sub-agentii, sandbox-ul `Docker` si tracing-ul. Directorul `ui` contine interfata cu utilizatorul: bucla principala, comenzile `slash`, sistemul de `skill-uri`, gestionarea sesiunilor si preferintelor. Directorul `web` contine un dashboard analitic `Flask` care vizualizeaza sesiunile, consumul de `tokeni` si activitatea uneltelor. Directorul `eval` contine un `runner` de evaluari cu `metrici` precum `BLEU`, `ROUGE` si `potrivire semantica`. Documentatia detaliata per-subsistem se gaseste in `docs`, iar configurarea implicita in `config/config.yml`.

```

vibe-cli/
|- config/
|- docs/
|- eval/
|- scripts/
|- src/
|- tests/
|- ui/
|- web/
|- Makefile
|- pyproject.toml
\-- ARCHITECTURE.md

config/
|- config.yml
\-- eval.yml

src/
|- main.py
|- tools.py
|- tool_guard.py
|- providers.py
|- permissions.py
|- sandbox.py
|- mcp_client.py
|- artifacts.py
|- tracing.py
|- compaction/
\-- subagents/

ui/
|- app.py
|- ui.py
|- commands.py
|- skills.py
|- sessions.py
|- preferences.py
|- provider_handler.py
|- pastes.py
\-- updater.py
```

Figura 2.3: Structura directoarelor vitale ale proiectului

¹`uv` este un manager de pachete pentru Python, similar cu `pip`, dar mult mai eficient

2.9 Configurare

Configurarea aplicatiei se face prin intermediul fisierului `config.yml` care contine catalogul de furnizori (Ollama, OpenAI, Claude, Bedrock, Gemini), modelele disponibile pentru fiecare, pipeline-ul de compactare, setarile de tracing, configuratia sandbox-ului sau a subagentilor si a uneltelor. Cheile API sunt declarate ca variabile de mediu (de exemplu `${OPENAI_API_KEY}`) si sunt rezolvate la pornire. Daca nici o cheie nu este setata si Ollama nu este accesibil, aplicatia solicita interactiv introducerea unei chei la pornire prin comanda `/model` sau `/key`.

De asemenea pentru procesul de evaluare al capabilitatilor folosit pentru teste de sanatate si regresie, se pot modifica setarile din fisierul `/.vibe-cli/eval/eval.yml`

Preferintele utilizatorului (provider-ul implicit, modelul selectat per provider, nivelul de efort, serverele MCP, listele de permisiuni) sunt persistate in `~/.vibe-cli/preferences.json` si supravietuiesc intre sesiuni. Sesiunile de conversatie sunt salvate ca log-uri de evenimente in `~/.vibe-cli/sessions/`, sortate per proiect (directorul de lucru).

2.10 Interfata cu utilizatorul si instructiuni de utilizare

Interfata serveste ca un mediu de legatura intre utilizator si agent, dar si ca un spatiu de configurare al aplicatiei. Aceasta apeleaza functii ale agentului in acelasi proces Python, comunicarea fiind prin apeluri de functii, nu prin retea.

In dezvoltarea interfetei, am pornit folosind solutia oferita de biblioteca Textual. La finalul primului prototip interfata arata modern si functiona corect. Insa, pe masura ce am populat conversatia cu asistentul, am observat ca functionalitatea de scroll a ferestrei nu functiona corect, sarind catre partea de sus a ecranului. Dupa ce m-am interesat mai atent la solutiile implementate de alte programe asemanatoare precum Claude Code, am observat ca acestea nu folosesc un TUI propriu zis, ci doar scriu in terminal folosind stdout, lasand astfel scroll-ul nativ al terminalului sa se ocupe de istoricul conversatiei. Am decis astfel sa abandonez solutia cu Textual si sa implementez o interfata bazata pe `prompt_toolkit` si `Rich` care sa scrie in terminal, oferind astfel o experienta mult mai fluida si fara probleme de scroll sau lag.

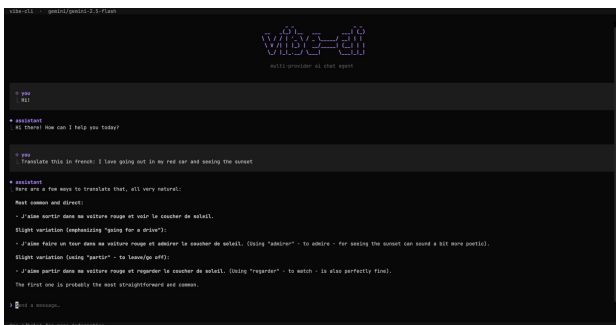


Figura 2.4: Prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual

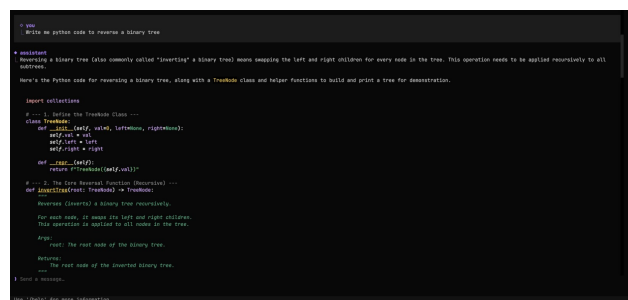


Figura 2.5: Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual

In final, a fost o decizie buna sa abandonez implementarea cu Textual, deoarece am reusit sa produc rezultate mai bune folosind terminalul clasic. Solutia finala este si mai estetica, cat si simpla, functionala si performanta.

Schimbarea nu s-a produs intr-un stadiu avansat al dezvoltarii, ci chiar de la primele prototipuri de interfata, lucru care nu a ingreunat cu mult procesul de dezvoltare.

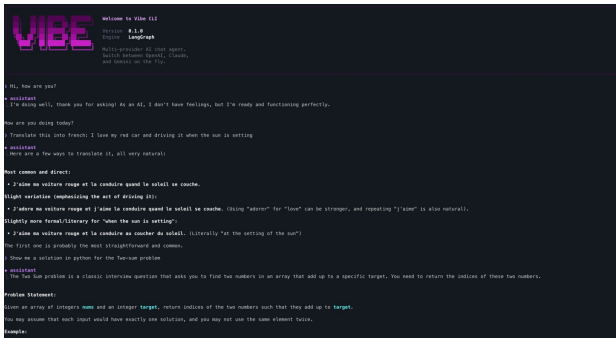


Figura 2.6: A doua versiune a interfetei, realizata fara biblioteca Textual

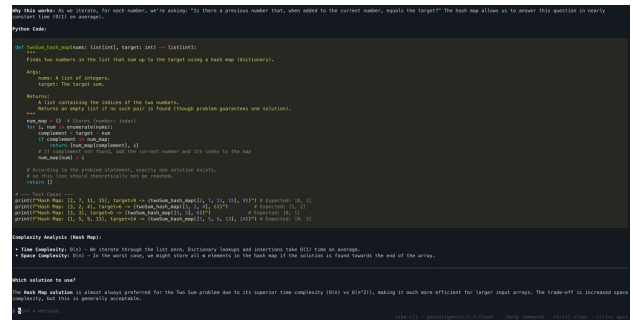


Figura 2.7: Continuarea conversatiei cu a doua versiune a interfetei, realizata fara biblioteca Textual

Dupa inlocuirea implementarii, am continuat sa adaug functionalitati pentru a imbogati experienta utilizatorului. Interfata ofera mai multe elemente vizuale menite sa informeze utilizatorul in timp real despre activitatea agentului:

- **Spinner animat cu eticheta de faza si cronometru:** textul spinner-ului reflecta faza curenta si masoara timpul scurs, de exemplu `thinking...` 1.2s in timpul inferentei LLM, sau `Read(README.md)...` 0.4s in timpul executiei unei unelte
- **Log persistent de unelte:** fiecare apel de unelte lasa o linie $\hookrightarrow$ `Tool(arg)` in istoricul vizibil, astfel incat si uneltele care se executa rapid sunt vizibile
- **Panouri de diff pentru write_file:** apelurile reusite de scriere sunt redactate ca un diff unificat intr-un panou Rich cu numarul de linii adaugate si sterse
- **Paste folding:** textele lipite mai lungi de 200 de caractere sunt inlocuite in prompt cu un placeholder scurt `[Paste #N <len> chars]` pentru lizibilitate, continutul original fiind restaurat inainte de trimitere
- **Footer de timp la sfarsitul fiecarui tur:** o linie care indica durata turului, cu o fraza aleasa aleator

Pe langa elementele vizuale, functionalitatile principale includ:

- Unelte inteligente de interactiune cu mediul virtual (create/editare/citire fisiere, cautare web, executare comenzi in terminal) care instruiesc modelul de limbaj sa le foloseasca intr-un mod inteligent din punct de vedere al consumului ferestrei de context.
- Gardarea apelului logic al uneltelor. Uneltele au anumite restrictii pentru a asigura folosirea corecta. De exemplu, modelul nu poate apela uneltele `edit_file` fara ca mai intai sa apeleze uneltele `read_file`. De asemenea sunt gardari de citire a fisierelor. Modelul nu poate citi un fisier intreg pentru a garanta ca nu isi va umple toata fereastra de context cu output-ul acelei unelte.
- Alegerea modelului din 5 provideri (Ollama, OpenAI, Claude, Bedrock, Gemini) si a efortului de gandire. Furnizorii de modele de limbaj nu sunt limitati si lista poate creste prin extinderea clasei de baza `BaseProvider`.
- Gestionarea sesiunilor cu posibilitatea de a salva, sterge, lista sau continua conversatii anterioare. Fiecare proiect include istoricul propriu al conversatiilor.

- Sistemul de permisiuni per-unealta cu liste de always-allow si always-deny persistente
- Conectarea cu servere MCP prin transport stdio, HTTP sau SSE
- Skill-uri bazate pe fisiere Markdown cu incarcare progresiva (doar numele si descrierea in prompt-ul de sistem, corpul incarcat la cerere)
- Sub-agenti specializati extensibili (cativa agenti de baza precum: general, code search, web research, dar pot fi adaugati mai multi sub-agenti prin extinderea clasei **Subagent**) care ruleaza in paralel cu modele configurabile per-agent.
- Pipeline de compactare a contextului cu mai multe stadii (trunchiere, sumarizare, deduplicare) configurabil.
- Sandbox Docker optional pentru executia izolata a comenzilor
- Tracing prin LangFuse Cloud pentru observabilitate
- Dashboard web analitic (Flask) cu vizualizare de metrice, cost si activitate
- Auto-completare pentru comenzile slash si actualizare automata a versiunii

Pentru instalarea aplicatiei, utilizatorul poate rula comanda de instalare cu o singura linie:

```
curl -fsSL https://raw.githubusercontent.com/Moshulika/vibe-cli/install.sh |
bash
```

Dupa instalare, comanda **vibe** este disponibila global din orice director. Pentru dezvoltare, se foloseste **make ui** din directorul proiectului. Dupa pornire, utilizatorul este intampinat de un ecran de start care ofera optiunea de a incepe o conversatie noua sau de a continua una veche.

Comenzile pot fi rulate in orice moment al conversatiei, si au formatul **/comanda** cu auto-completare la apasarea tastei Tab. De asemenea exista si scurtaturi rapide pe tastatura, precum **CTRL+L** pentru a sterge conversatia si a incepe una noua, si **CTRL+C** pentru a inchide aplicatia.

Comanda	Descriere
/help	Listarea tuturor comenzilor si a scurtaturilor de tastatura
/model	Selectarea provider-ului si modelului, sau setarea cheii API
/key	Setarea sau actualizarea cheii API per provider
/effort	Selectarea nivelului de efort al modelului (low/medium/high)
/mcp	Configurarea serverelor MCP (adaugare/stergere/activare)
/skills	Listarea, activarea sau reincarcarea skill-urilor
/subagents	Listarea tipurilor de sub-agenti si configurarea modelelor per tip
/plan	Activarea/dezactivarea modului de planificare
/agent	Activarea/dezactivarea modului agentic
/permissions	Vizualizarea si editarea listelor de permisiuni per unealta
/clear	Stergerea conversatiei curente si rotirea la o sesiune noua
/sessions	Listarea sesiunilor salvate pentru proiectul curent
/resume	Continuarea ultimei sesiuni sau a uneia specifice prin ID
/compaction	Inspectarea pipeline-ului de compactare si selectarea modelului
/compact	Fortarea unei compactari imediate a contextului
/tracing	Configurarea si activarea tracing-ului LangFuse
/score	Evaluarea ultimului tur in LangFuse (good/bad)
/dataset	Adaugarea ultimului tur la un dataset LangFuse
/update	Actualizarea la ultima versiune din git
/exit	Salvarea conversatiei si inchiderea aplicatiei

Tabela 2.4: Comenzi disponibile in interfata

2.11 Management-ul contextului si al memoriei

2.11.1 Memoria

2.11.2 Contextul

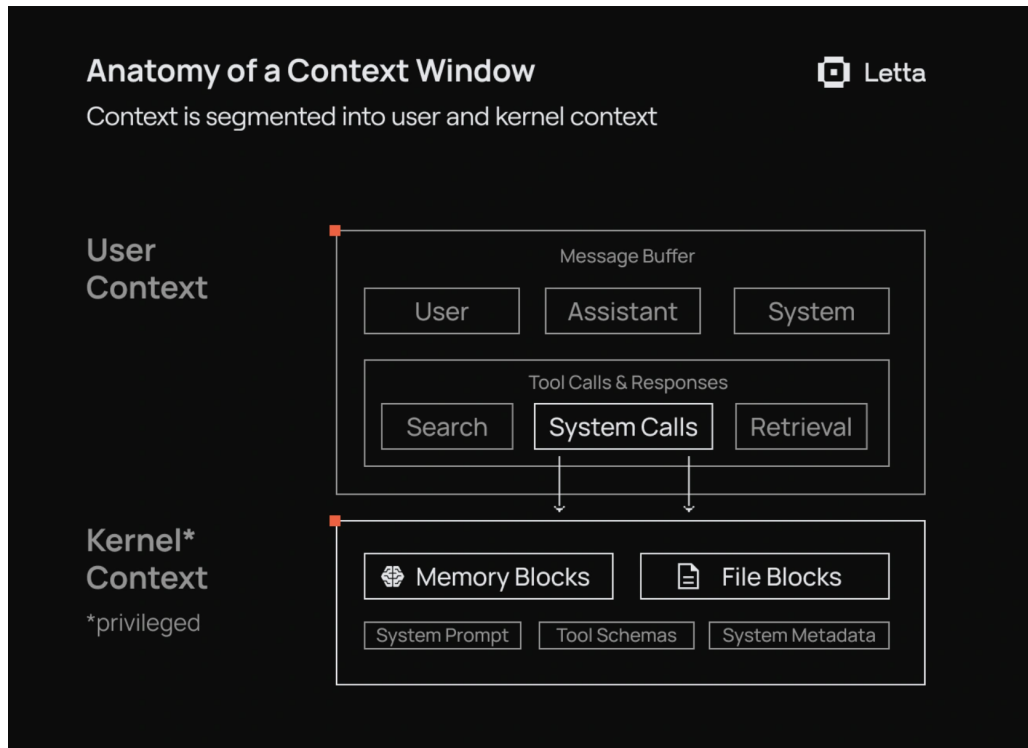


Figura 2.8: Impartirea ferestrei de context (sursa)

2.12 Subagenti, Unelte si capabilitati

2.12.1 Tools

2.12.2 Skills

2.12.3 Subagents

2.13 Extensibilitate

2.13.1 MCP

2.13.2 Subagenti si arhitectura agnostica de furnizori

2.14 Securitate

2.14.1 Sandboxing

2.14.2 Guardrails, OOS and dangerous intent

2.14.3 Tool execution order and logic

2.15 Observabilitate

2.15.1 Langfuse

2.15.2 Panou web

Tabela 2.1: Produse AI ale principalilor furnizori de modele de limbaj

Companie	Produs	Descriere	Audienta	Pret
Anthropic	Claude (web/app)	Asistent AI conversational pentru analiza, scriere, programare si rationament, cu suport multimodal si artefacte interactive	Utilizatori generali, profesionisti	Freemium (Pro: \$20/luna, Max: \$100–200/luna)
	Claude Code	Agent de programare agentic care ruleaza in terminal, IDE sau Slack; poate naviga coduri mari, crea PR-uri si rula teste automat	Dezvoltatori software	Inclus in Pro/Max (API: cost per token)
	Claude Cowork	Agent agentic de productivitate care lucreaza cu fisiere locale si aplicatii cloud (Slack, Google Drive) pentru a automatiza sarcini recurente	Profesionisti, echipe	Inclus in Pro/Max
	Claude Design	Instrument de prototipare vizuala integrat in Claude, care genereaza interfete UI interactive si mockup-uri pe baza descrierilor in limbaj natural	Designeri, dezvoltatori	Inclus in Pro/Max
Google	Gemini (web/app)	Asistent AI conversational multimodal cu acces la cautare Google, extensii si generare de imagini; disponibil pe web, Android si iOS	Utilizatori generali	Freemium (Advanced: \$19.99/luna in Google One AI Premium)
	Gemini CLI	Agent de programare open-source (Apache 2.0) care ruleaza in terminal, cu suport MCP, cautare Google si context de 1M tokeni	Dezvoltatori software	Gratuit (60 req/min, 1000 req/zi cu cont Google)
	NotebookLM	Asistent de cercetare alimentat de Gemini care analizeaza documente incarcate (PDF, site-uri, video-uri) si genereaza rezumate, podcast-uri audio si conexiuni intre surse	Cercetatori, studenti	Gratuit (Plus: \$7.99/luna, Business: contact)
	Vertex AI	Platforma enterprise pentru constructia, antrenarea si implementarea de agenti AI si modele ML, cu acces la 200+ modele si instrumente MLOps	Echipe enterprise, ingineri ML	Platit (pay-as-you-go, de la \$0.0001 per operatie)
	Firebase Studio	Mediu de dezvoltare cloud (IDE) cu agenti AI integrati pentru constructia de aplicatii full-stack (backend, frontend, mobil) direct din browser	Dezvoltatori software	Gratuit in previzualizare (3 workspace-uri; se include in 2027)
OpenAI	ChatGPT (web/app)	Asistent AI conversational cu suport multimodal, cautare web, generare de imagini, analiza de date si plugin-uri; disponibil pe web, iOS, Android si desktop	Utilizatori generali, profesionisti	Freemium (Plus: \$20/luna, Pro: \$200/luna)
	Codex	Agent de inginerie software cloud care executa sarcini de programare in paralel, in sandbox-uri izolate preincarcate cu codul din repository-ul GitHub	Dezvoltatori software, echipe	Inclus in ChatGPT Pro/Business/Enterprise (API: \$1.50/\$6 per MTok)
Microsoft/ GitHub	GitHub Copilot	Agent de programare integrat in GitHub si disponibil ca aplicatie CLI, extensie IDE (VS Code, JetBrains) si in github.com; ofera completare de cod, chat, revizuire PR-uri si rezolvare automata de issue-uri	Dezvoltatori software, echipe	Freemium (gratuit pentru repo-uri publice; Pro: \$10/luna, Business: \$19/luna, Enterprise: \$39/luna)

Tabela 2.3: Hardware recomandat pentru modele open-source

Model	Parametri	GPU Recomandat	VRAM	RAM
LLaMA 3	70B	2x NVIDIA A100 80GB sau 2x RTX 4090	40–160 GB (cuantizat vs FP16)	64–128 GB
Mistral	7B	NVIDIA RTX 3060 / RTX 4090	4–16 GB	8–32 GB
Qwen 2.5 (14B)	14B	NVIDIA RTX 3090 / RTX 4090	16–24 GB	16–64 GB
Gemma 4 (E4B)	4.5B efectivi (8B total)	NVIDIA RTX 3060 / RTX 4060	4–16 GB (cuantizat vs BF16)	8–32 GB

Capitolul 3

REZULTATE EXPERIMENTALE

- 3.1 Experimente, evals si benchmarking
- 3.2 Contextul in mediul agentic
- 3.3 Modele vs context si unelte

Capitolul 4

CONCLUZII

Capitolul 5

IMBUNATATIRI VIITOARE

5.1 Automatizari, integrari cu alte programe

BIBLIOGRAFIE

- [1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.
- [3] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. version: 1.
- [5] Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36:10088–10115.
- [7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35:24824–24837.
- [9] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.
- [11] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models.
- [12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. 12:157–173.

- [13] Kiran Vodrahalli, Santiago Ontanon, Nilesh Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. version: 2.
- [14] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
- [15] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain.
- [16] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 86(11):2278–2324.
- [17] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. 9:1735–1780.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- [19] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners.
- [20] Yichun Xu, Navjot K. Khaira, and Tejinder Singh. KV cache optimization strategies for scalable and efficient LLM inference. version: 1.
- [21] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.
- [22] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate.
- [23] Zhongmou He, Yee Man Choi, Kexun Zhang, Ivan Bercovich, Jiabao Ji, Junting Zhou, Dejie Xu, Aidan Zhang, Yixiao Zeng, and Lei Li. HARDTESTGEN: A high-quality RL verifier generation pipeline for LLM algorithmic coding.
- [24] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncarencu, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques.

- [25] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines.
- [26] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634. Association for Computational Linguistics.
- [27] Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing.
- [28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.